claude-windows / 144a31b7-7962-4fee-9d6f-8a7854daa63f

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\144a31b7-7962-4fee-9d6f-8a7854daa63f\subagents\workflows\wf_134043cd-8c3\agent-ae904a570f767e974.jsonl`
- SHA-256: `f9de4b133f26faaecb4c7712ab115dbde4f1c412f50579068e46fceb64505ec2`
- Source modified: `2026-06-20T09:11:44+00:00`
- Imported at: `2026-07-05T16:48:28+00:00`
- project: `wf_134043cd-8c3`
- session_id: `144a31b7-7962-4fee-9d6f-8a7854daa63f`

Transcript

1. user (2026-06-20T09:02:57.843Z)

Design the KhelSutra "locally installed, UI-powered badminton rally indexer + reel generator" ? a SINGLE-BOX appliance where the web UI drives the full upload -> index -> pick rallies -> reel -> download loop ? using this lens:
DECOUPLED-PROFILE APPLIANCE: the UI/FastAPI enqueues jobs to the StorageBackend bucket; the worker (`python -m backend.worker`) processes them ? co-located on the same box for the CPU/gemini case, OR on a remote GPU box for the BYO-GPU case ? and the SAME UI works for both by watching job/output state in storage. Optimize for "one UI, compute placement is a setting" (the BYO-compute story the request-access form already probes).

Owner's framing: both the website and the engine codebase now live on ONE box (already true on the droplet); augment the UI to also DRIVE the badminton-highlight-indexer so the whole thing operates as a self-contained locally-installed appliance.

GROUND EVERY DECISION in this verified research (decoupled-compute architecture, engine API/worker, hosting/exposure, UI repo). Reconcile with the decoupled architecture ? do NOT propose something that fights the StorageBackend/worker model. Be brutally honest about the CPU-only droplet (no local WASB), the no-auth API, and the shared-testbed boundaries:


```
{
  "decoupled": {
    "storage_contract": "StorageBackend ABC = backend/storage/base.py:17. Abstract methods: fetch_to_local(ref, dst=None, *, overwrite=False)->str (base.py:21), put(local_path, ref, *, content_type=None)->StorageRef (:27), exists (:32), stat (:35), list(prefix)->Iterator[StorageRef] (:38), delete (:42); plus a NON-abstract signed_url(ref,*,ttl,method) that raises NotImplementedError by default (:45-49) ? callers MUST treat that as expected and fall back to fetch_to_local (local can't sign; gdrive only emulates).\n\nURI scheme (backend/storage/ref.py:48 parse_uri + docstring :1-13): bare path or local:// => local (today's behaviour, IDENTITY passthrough, no copy); s3://<bucket>/<key> => s3 (covers AWS + Cloudflare R2 + DO Spaces + MinIO, distinguished only by endpoint_url); gcs:// (or gs://, normalized) => gcs; gdrive://<folderId>/<path> => gdrive (follow-up stub). Windows C:\\... has no :// so falls through to local. Value objects: StorageRef(backend,bucket,key,uri) frozen dataclass with a .name property (ref.py:34-45); StorageStat(size,modified,etag,content_type) (ref.py:25).\n\nThin URI helpers in backend/storage/__init__.py: get_storage(backend,config,**overrides) (:43), fetch(uri,dst,config,overwrite) (:55), put(local_path,uri,config,content_type) (:63), list_uris(prefix_uri,config) (:71). Backends are lazy-resolved (__init__.py:26 _backend_class) so `import backend.storage` never pulls boto3/google SDKs ? they import inside the concrete module (s3.py:29 `import boto3`). CREDENTIALS NEVER IN CONFIG: s3.py:8-10 + S3Storage.__init__ (s3.py:22) takes only endpoint_url/region/addressing_style/signed_url_ttl; boto3's default chain (env AWS_ACCESS_KEY_ID/SECRET, instance role, ~/.aws) supplies creds. A client can be injected for tests (s3.py:26).\n\nLocalStorage (local.py:18): fetch_to_local is identity when dst is None or dst==src (local.py:33), else shutil.copyfile; put is a copy that no-ops when src==dst (:42-47); list walks the dir (:56). So backend='local' is byte-for-byte the pre-storage-layer behaviour.\n\nBucket layout (04 doc §3, lines 56-66): videos/<md5>.mp4 (immutable, byte-
```

identical to what labels were made on), labels/<sport>/<video_id>.csv, manifest.json (the join table ? needs an md5 field added, an M2 prereq per 02 §4), weights/<semver>/{model,manifest} + weights/checkpoints/, outputs/<video_id>/{highlights.mp4, intervals.csv, report.json}.\n\nStorageConfig (backend/config/models.py:291): backend: Literal[\"local\",\"s3\",\"gcs\",\"gdrive\"]=\"local\"; output_root:str=\"\"; per-backend loose dicts local/s3/gcs/gdrive (:301-304) passed straight to the backend constructor (endpoints/regions only, no secrets).",

"worker_cli": "Entrypoint backend/worker/__main__.py ? `python -m backend.worker {infer|train}`. Docstring (:1-15): a THIN ORCHESTRATOR composing the SAME building blocks as the backend.main CLI (validator registry -> segmenter -> report), never a forked pipeline. Parser at build_parser() (:302).\n\nINFER ? cmd_infer (:88): `python -m backend.worker infer --video-uri <uri> --out-uri <prefix> [--segmenter] [--config] [--scratch DIR] [--max-frames] [--set k=v]`. Flow: (1) load_config + _apply_overrides (:49, dotted-path setattr onto the typed pydantic config, _coerce bool/int/float at :36); set config.output.output_dir=scratch (:95). (2) PULL: storage.fetch(video_uri, scratch/in.mp4) ? identity for local://, download for s3/gcs (:98). (3) IDENTITY: compute_video_id = whole-file MD5 (:101), the labels<->video join key; pulled bytes must be byte-identical. (4) VALIDATE via the SAME validator_registry as the main CLI (:105), write video row to a scratch SQLite DB (:107). (5) SEGMENT: segmenter_registry.get(seg_name or config.indexing.default_segmenter).process_video(...) (:120-127). (6) emit_indexer_report in a finally (:136, parity with main CLI, never raises). (7) SERIALIZE outputs/<video_id>/{intervals.csv, report.json} ? intervals.csv is decimal-second [start,end)+shot_count+ending_reason, the join-friendly schema matching the rally-annotator labels (_write_intervals_csv :62). (8) PUSH each file via storage.put to <out_prefix>/outputs/<video_id>/<fn> (:149-154), idempotent on video_id.\n\nTRAIN ? cmd_train (:192): `python -m backend.worker train --corpus-uri <prefix> --out-uri <prefix> --version <semver> [--trajectory-source synth|detector] [--segmenter wasb_hybrid] [--floor] [--fps] [--frame-width] [--config] [--scratch] [--set]`. Flow: list labels under <corpus>/labels/*.csv (storage.list_uris :217); REQUIRES >=2 for leave-one-video-out (:219). Two trajectory sources, train pipeline+harness identical either way (docstring :196-201): (a) synth (DEFAULT, the CPU \"mock GPU\") ? synth_trajectory_from_labels (stub_runner) makes deterministic label-consistent shuttle paths, NO GPU/WSL (:260-262); (b) detector ? REAL trajectories: resolve a DetectorRunner once (_resolve_train_detector :162), index videos/ by stem, fetch each video, probe real fps/width, runner.run_predict -> trajectory CSV (:242-258). Build manifest.json of specs (:269), then SHELL OUT (subprocess, because backend must NOT import training) to `python -m training.gen0.harness --manifest <m> --out <dir> --version <semver> [--floor <f>]` (:276-283). PUSH the versioned bundle to <out_prefix>/weights/<version>/<fn> (:289-294).\n\n_resolve_train_detector (:162) reuses the segmenter's _resolve_runner seam so worker and live CLI build the detector identically; rejects a non-trajectory segmenter (:179) and runs runner.healthcheck() before use (:184). Config knobs: --config (config.json), --set dotted overrides; the validated GPU-free/secret-free combo is RALLY_DETECTOR_IMPL=stub + --set indexing.skip_ai_handoff=true + storage.s3 settings (docstring :8-11, HANDOFF.md).",

"compute_abstraction": "Two ORTHOGONAL axes, deliberately separated (07 doc §2, lines 27-39): (A) ComputeBackend = WHERE a job runs (coarse, per-job: local | hosted | byo_worker / DO droplet / serverless) ? picks the machine; (B) DeviceContext = HOW a GPU stage runs ON that machine (fine, spec-aware, per-stage: probe the real GPU, resolve batch/precision/chunk knobs). Conflating them is the named design trap.\n\nSTATE TODAY: ComputeBackend is PLANNED ONLY ? `grep ComputeBackend backend/` returns nothing; it is named in PLATFORM_ARCHITECTURE.md (registry local|hosted|byo_worker) and inventoried as a FUTURE registry alongside StorageBackend in 07 §1 (:19) and 01 §3.4. NO ComputeBackend code, NO DeviceContext/DeviceSpec/DeviceProfile code ? 07 doc is a PROPOSAL (status :3) to be built at M3.5, NOT M0 (07 §6 :153, §5 caveat 2 :150).\n\nThe compute MAP / GPU-vs-CPU contract (07 §3 table lines 45-56, the load-bearing rule :57-59): \"a GPU-bound stage is a pure, restartable unit whose only job is frames -> artifact (the Frame,Visibility,X,Y CSV or a feature JSON); everything downstream is device-agnostic CPU that reads that artifact.\" GPU work is concentrated in the detection substrate ONLY ? person detection (torchvision, clean CPU fallback) + shuttle trajectory (WASB torch / TrackNet TF). Validators, ffmpeg chunk/stitch (libx264, nvenc optional), ByteTrack, windowing/candidate-merge, Gemini handoff (network I/O), DB/eval/calibration, Gen-0 head training (~4.5GB, minutes) are CPU/device-agnostic. This artifact boundary is EXACTLY where GPU and CPU work can live on different machines (the decoupling), and is already how EXPERIMENT_HARNESS/GOLDEN_DATA split today (07 §3 point 2).\n\nWhat abstracts compute placement TODAY (built): (1) the storage URI scheme ? local vs s3/gcs decides whether bytes are co-located or remote, transparently to the pipeline; (2) the detector_impl seam in trajectory_hybrid.py::_resolve_runner (:65): RALLY_DETECTOR_IMPL env (precedence) or indexing.detector_impl in {auto=WSL via _build_runner (:86), stub=CPU mock StubDetectorRunner (:77-81), native=Linux-native via _build_native_runner (:82-85)}. The native branch is the M3.5 hook but the base _build_native_runner REFUSES with

NotImplementedError (\"only WASB has a Linux-native runner. Use detector_impl='auto' (WSL) or 'stub'\", trajectory_hybrid.py:60-63) ? so LOCAL GPU = WSL today (auto), REMOTE/Linux GPU box = native (not yet built), CPU/CI = stub. The proposed DeviceContext layer (DeviceSpec/DeviceKnobs/DeviceProfile registry, 07 §4-5) would replace scattered device= logic and fix WASB's hardcoded device='cuda' with no CPU fallback (the one inconsistency, 07 §1 :16) ? but it only bites once the native runner lands, hence deferred to M3.5.",

"built_vs_next": "BUILT & MERGED to master (PRs #246-#254, suite 936 green per HANDOFF.md:9): the full bucket-driven train->infer pipeline running E2E on a CPU box with the GPU MOCKED (deterministic stub), validated on a DO droplet reading/writing DO Spaces:\n- Storage layer (#249): backend/storage/ ? local + S3 (AWS/DO Spaces/R2/MinIO) + GCS, URI-driven; gdrive is a stub.\n- Worker dispatcher (#251 infer, #254 train): backend/worker/__main__.py ? pull -> SAME pipeline -> push.\n- CPU-mock detector seam (#247): backend/pipeline/detectors/stub_runner.py via RALLY_DETECTOR_IMPL=stub / indexing.detector_impl=stub.\n- Cross-platform canonicalization (#252); GPU-box provisioning scripts (#253): deploy/digitalocean/{bootstrap,mount_scratch,gpu_setup}.sh (NVMe scratch + native WASB conda env setup) ? scripts exist but the native runner that uses them does not.\n- Plan directory (#246): docs/DECOUPLED_COMPUTE/.\nVerified E2E (CPU, no GPU, no Gemini key, HANDOFF.md:26-30): worker train pulled 7 labels from Spaces -> synth_trajectory_from_labels (label-consistent CPU mock) -> shelled to training.gen0.harness -> LOVO over 7 videos (F1 1.000 on the mock, gate=no-ship by design) -> weights/ to Spaces; worker infer pulled a real video -> stub -> outputs/<md5>/ to Spaces.\n\nNEXT BUILD = M3.5, the ONE remaining piece for real (non-mock) training/inference (HANDOFF.md:50-65, 04 doc §M3.5 :98): build NativeWasbRunner(DetectorRunner) in backend/pipeline/detectors/ ? sibling to WasbRunner but WITHOUT WslCommandMixin: run WASB in-process/native (no wsl / conda activate / to_wsl_mnt_path), device cuda/mps/cpu, weights via env (not ~/models), emit the IDENTICAL Frame,Visibility,X,Y CSV. Wire it into the existing native branch by overriding _build_native_runner in the WASB segmenter (today the base refuses, trajectory_hybrid.py:57-63). Acceptance gate = byte-parity: same clip -> WSL CSV (Windows) == native CSV (Linux) + identical candidate windows. Then wire a real-extract path into worker train/infer (--trajectory-source detector already exists; only the trajectory source flips, harness+push unchanged). Also proposed-not-built at M3.5: the DeviceContext/DeviceSpec/DeviceProfile device layer (07 doc) and the ComputeBackend registry. STILL NOTHING: no Dockerfile anywhere (README §96 verification), no cost ledger / cost columns on the jobs table (05 doc), no remote inference endpoint / API-key-gated serverless, no bucket-triggered retrain flywheel. Gen-2 VLM fine-tune is explicitly OUT OF SCOPE for the $200 budget.",

"single_box_fit": "There is NO literal \"single-box profile\" named in these docs ? the architecture is a control-plane / data-plane / compute-worker split (04 doc §1 :12-27), and a SINGLE BOX is simply the DEGENERATE configuration of that same split where all three planes collapse onto one machine. The decoupling docs frame the END STATE (separate rented GPU + remote bucket); the single-box case is what already runs.\n\nHOW it maps, concretely:\n- STORAGE = LOCAL. Set storage.backend='local' (the DEFAULT, config/models.py:298). Then storage.fetch is an identity passthrough (no copy ? local.py:33) and storage.put no-ops when src==dst (local.py:42-47); parse_uri treats bare/Windows paths as local (ref.py:48-57). The worker's pull->pipeline->push collapses to \"read local file -> run -> write local dir\", byte-for-byte the pre-storage behaviour. FULLY BUILT, and the default.\n- WORKER IN-PROCESS / CO-LOCATED. `python -m backend.worker {infer|train}` runs the SAME registries as backend.main in the SAME process (worker docstring :12-15); the control plane (the shipped async jobs table + ThreadPoolExecutor on the owner's box, PLATFORM_ARCHITECTURE.md:60,475 ? single-tenant, local-only) and the compute worker are the same machine. The owner's CURRENT Windows/WSL dev box IS a single box: control plane + storage(local) + worker + GPU(WSL) all co-located. SUPPORTED today.\n- COMPUTE on a single box. The detector_impl seam picks the local detector: auto=WSL (the owner's GPU today) or stub=CPU. No ComputeBackend needed because there is only one machine ? ComputeBackend (WHERE) is the thing you ADD when the GPU moves off-box; on a single box it is identically the no-op \"local\" choice that need not exist yet (07 §2, 01 §3.4).\n- The DO CPU droplet 168.144.126.176 is ALREADY a working single-box instance of this in the cloud: BOTH /srv/khelsutra-site AND ~/rally (engine) live on that ONE box, and it ran the full train+infer E2E (GPU mocked + storage pointed at Spaces). Swap storage.s3->local and that same box is a pure single-box deployment.\n\nWHAT a single box ALREADY supports: the entire infer + Gen-0 train flow, end-to-end, mocked-GPU, on CPU; and on the owner's Windows+WSL box the REAL WSL detector via detector_impl=auto. WHAT IS MISSING for a single box to do REAL training/inference WITHOUT WSL (i.e. a single Linux GPU box): the M3.5 NativeWasbRunner ? until it lands, a non-Windows single box can only run stub (CPU mock) or must shell to WSL (Windows-only). Everything else a single box needs (storage=local default, worker, validators, Gen-0 harness, ffmpeg) is built. The cost-ledger / API-key / serverless / Dockerfile gaps are about MULTI-tenant remote serving, not the single-box path ? a single box does not need them.",

```
    "file_refs": [
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\docs\\DECOUPLED_COMPUTE\\README.md",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\docs\\DECOUPLED_COMPUTE\\HANDOFF.md",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\docs\\DECOUPLED_COMPUTE\\01-EXISTING-PLANS-AND-GAP.md",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\docs\\DECOUPLED_COMPUTE\\02-COMPUTE-MAP-AND-CONTRACT.md",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\docs\\DECOUPLED_COMPUTE\\03-PLATFORM-OPTIONS.md",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\docs\\DECOUPLED_COMPUTE\\04-BUILD-AND-TEST-PLAN.md",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\docs\\DECOUPLED_COMPUTE\\07-COMPUTE-ABSTRACTION.md",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\worker\\__main__.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\storage\\__init__.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\storage\\base.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\storage\\ref.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\storage\\s3.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\storage\\local.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\config\\models.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\pipeline\\segmenters\\trajectory_hybrid.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\pipeline\\detectors\\stub_runner.py"
    ]
  },
  "engineApi": {
    "process_flow": "FULL LOOP today (single FastAPI box, started via `python -m backend.main
--server` -> main.py:632-638 uvicorn on 127.0.0.1:8000). Step by step:\n\n1. UPLOAD (optional,
browser-side; chunked island in backend/api/uploads.py). A browser file is uploaded in
sequential parts <= security.max_part_mb (~95MB, edge-proxy cap): POST /api/upload/init
(uploads.py:64) validates extension (mp4/mov) + size, opens a `.partial-<id>` temp under
security.upload_dir (output/uploads) and returns upload_id + next_part URL; PUT
/api/upload/{id}/part/{i} (uploads.py:89) appends raw bytes, parts STRICTLY sequential (409 on
gap), enforces per-part + whole-file caps (413 aborts); POST /api/upload/{id}/complete
(uploads.py:123) verifies part count, os.replace into upload_dir/<filename> (de-dupes name
collisions), computes video_id = MD5 (compute_video_id) and returns {path (absolute), video_id,
size_bytes, next:\"POST /api/process with this path\"}. DELETE /api/upload/{id} aborts + deletes
partial. Upload state is IN-PROCESS only (uploads.py:_uploads dict + lock) -> a restart drops
in-flight uploads (by design, single-box alpha). NOTE: the local /static UI does NOT use these
endpoints; it just takes a server-local path string.\n\n2. PROCESS submit (main.py:240 POST
/api/process, status_code=202). Fast-fail checks run synchronously in the request:
validate_input_path (+ optional security.allowed_video_root jail), os.path.exists -> 404,
unknown segmenter -> 400. Then it persists a job row in state 'queued' (db.create_job,
main.py:264) and fires `_get_executor().submit(_drain_due_jobs)`; returns 202 {job_id,
status:\"queued\", poll:\"/api/jobs/{id}\"}. ALL slow work ? including MD5 hashing of the file ?
happens on the worker thread (#16), not in the request.\n\n3. INDEX (worker thread,
_execute_processing main.py:125). Stages surfaced via progress callback (hashing -> validating
-> segmenting(<name>) -> finalizing): compute_video_id; db.get_video to detect a re-ingest;
registry.run_all_with_context runs the pluggable validators (resolution/fps/blur/scene-
cuts/court-relevance from config.validation). On any validation failure -> upsert video status
'failed', return failed result (no destruction of prior good segments). On pass -> upsert
'processed', resolve segmenter class from segmenter_registry, record pre-run watermarks
(db.segment_watermarks), run segmenter.process_video(video_id, path, player_pool, max_frames).
Destroy-AFTER-confirm: on non-empty result prune the prior rows, on empty/raise restore prior
rows (_finalize_reingest main.py:107). A per-video observability report is always emitted in
finally: (emit_indexer_report) and grafted as result[\"reports\"]. Worker records terminal state
via db.mark_job_done (result carries the pipeline verdict in result[\"status\"]) or
db.mark_job_failed.\n\n4. POLL (main.py:274 GET /api/jobs/{job_id}). Returns {job_id, status
(queued|running|waiting|done|failed = EXECUTION state), progress, video_id, error, result,
reports, created/started/finished_at}. UI loops every 3s until status done|failed; the
pipeline's own success/failed-validation verdict lives in result[\"status\"], distinct from job
status.\n\n5. QUERY (main.py:331 POST /api/query) -> db.query_play_segments(video_id,
```

{server,receiver,duration_min,event_type}) -> {play_segments:[...]} (each with id, start_time, end_time, server, receiver, events, metadata.shot_count).\n\n6. COMPILE reel (main.py:342 POST /api/compile {video_id, segment_ids[], output_filename}). Looks up the video, intersects requested segment_ids with stored segments, applies stitching.min_shot_count floor (unknown shot_count exempt), builds (start,end) pairs, runs VideoCompiler(output_dir, begin/end_padding, watermark) -> writes reel into output_dir; returns {status:\"success\", filepath}. safe_output_filename rejects path traversal.\n\n7. DOWNLOAD reel (main.py:307 GET /api/highlights/{video_id}/{filename}) -> streams the compiled file from output_dir via FileResponse (video/mp4 or video/quicktime). The /static UI builds this URL after a successful compile to offer a download link.\n\n(There is ALSO a fully-decoupled path the FastAPI does NOT use: `python -m backend.worker {infer|train}` pulls inputs from the StorageBackend (backend/storage/), runs the SAME pipeline, pushes results back ? that is the engine-decoupling track, separate from this FastAPI loop.)",
    "endpoints": [
      "GET / (main.py:52) -> serves backend/api/static/index.html (the SPA shell)",
      "GET /api/config (main.py:121) -> returns the live typed AppConfig (global_config) as JSON",
      "POST /api/process (main.py:240, 202) -> req {video_path, player_pool?, max_frames?, segmenter?}; fast-fail path/exists/segmenter checks, creates 'queued' job, kicks a drain; resp {job_id, status:'queued', poll}",
      "GET /api/jobs/{job_id} (main.py:274) -> resp {job_id, status(queued|running|waiting|done|failed), progress, video_id, error, result, reports, created/started/finished_at}; 404 unknown",
      "POST /api/query (main.py:331) -> req {video_id, server?, receiver?, duration_min?, event_type?}; resp {play_segments:[...]}",
      "POST /api/compile (main.py:342) -> req {video_id, segment_ids[], output_filename}; runs VideoCompiler with stitching paddings+watermark; resp {status:'success', filepath}; 404 no video / 400 no matching segs or all below min_shot_count",
      "GET /api/highlights/{video_id}/{filename} (main.py:307) -> streams compiled reel from output_dir (FileResponse); 404 if not compiled yet",
      "POST /api/upload/init (uploads.py:64) -> req {filename, total_size_bytes?}; validates ext/size; resp {upload_id, max_part_mb, next_part}",
      "PUT /api/upload/{upload_id}/part/{index} (uploads.py:89) -> raw bytes body, strictly sequential parts; resp {received_parts, received_bytes}; 409 out-of-order, 413 over-cap (aborts)",
      "POST /api/upload/{upload_id}/complete (uploads.py:123) -> req {total_parts}; assembles file; resp {path(absolute), video_id, size_bytes, next}",
      "DELETE /api/upload/{upload_id} (uploads.py:147) -> aborts in-flight upload + deletes partial; resp {status:'aborted'}"
    ],
    "job_worker": "In-process async worker, backend/api/job_worker.py (re-exported on backend.main so monkeypatch seams work). ONE module-global ThreadPoolExecutor with max_workers=1 ON PURPOSE (job_worker.py:30-38): a single self-hosted box serializes GPU work and the Gemini provider is rate-fragile under parallel uploads ? so jobs run STRICTLY ONE AT A TIME (segmenters parallelize their own AI handoff internally via ai_max_workers). NO central scheduler: the DB jobs table IS the clock. Submission (/api/process and server startup) just calls _get_executor().submit(_drain_due_jobs). _drain_due_jobs (job_worker.py:100) is one tick: loop db.claim_due_job() (atomic compare-and-set on (id,status) flipping queued|due-waiting -> running, earliest-due first, so concurrent workers never double-claim) -> _run_claimed_job for each until nothing runnable. _run_claimed_job (job_worker.py:72) reconstructs the ProcessRequest from the row, runs _execute_processing with a progress cb that writes db.set_job_progress, then db.mark_job_done(result) (terminal 'done' = worker finished; pipeline verdict is in result['status']) or db.mark_job_failed on a raise. WAIT_AND_RESUME hook: if the pipeline raises JobWaiting(delay_sec) (job_worker.py:41) the job is parked via db.set_job_waiting (status 'waiting' + next_poll_at + progress) and re-claimed when due ? but NO production segmenter raises this today, so it is dormant. Self-scheduling: after each drain, _schedule_next_drain_if_waiting (job_worker.py:124) computes db.seconds_until_next_due and _arm_wake_timer arms ONE daemon threading.Timer (clamped to _MAX_DRAIN_WAKE_SEC=60s) to re-submit a drain, so parked work resumes with no further client request. CRASH RECOVERY (main.py:556 + db.fail_stale_jobs job_worker n/a -> database.py:613): at startup any job left in 'queued' OR 'running' from a dead process is swept to 'failed' (the executor was in-process, so it's truly dead) ? pollers see a terminal state instead of a hang. CAVEAT: fail_stale_jobs sweeps only queued/running, NOT 'waiting'; startup also fires one _drain_due_jobs to reclaim due waiting jobs + re-arm the wake timer (main.py:637), but a 'waiting' job whose timer died and is not yet due relies on that startup drain + the next submit to get re-armed. Single-process only:

all this state (executor, timer, uploads dict) is per-process ? there is no multi-box coordination beyond the DB row-claim.",
    "segmenter_selection": "SELECTION: per-request `segmenter` field (POST /api/process) wins; else config default. config.json sets indexing.default_segmenter='gemini' (config.json:26) [NOTE: the Pydantic built-in default in backend/config/models.py:122 is 'yolo_hybrid', but config.json overrides it to 'gemini']. main.py:150 reads getattr(global_config.indexing,'default_segmenter','motion'). Registered names (backend/pipeline/segmenters/__init__.py): motion, gemini, yolo_hybrid, tracknet_hybrid, wasb_hybrid, fusion_hybrid (each self-registers via segmenter_registry.register). Unknown name -> 400 at submit (main.py:256) or defensive 'failed' in-worker (main.py:172).\n\nWHAT EACH NEEDS:\n- gemini (gemini.py): pure cloud. Requires GEMINI_API_KEY env (gemini.py:86-95; resolved as <PROVIDER>_API_KEY). No GPU. Uploads re-encoded chunks to Google. This is the config.json default.\n- motion: pure-CPU OpenCV motion heuristic; no GPU, no key.\n- yolo_hybrid / tracknet_hybrid / wasb_hybrid / fusion_hybrid: two-stage trajectory hybrids (TrajectoryHybridSegmenter, trajectory_hybrid.py). Stage 2 (shuttle/person trajectory -> candidate windows) needs a DETECTOR RUNNER; Stage 3 is the AI handoff (Gemini) for semantic boundaries, which needs GEMINI_API_KEY UNLESS indexing.skip_ai_handoff=true (config models.py:196), in which case candidate windows ARE the rallies ? no provider, no key, nothing uploaded (the pure `gemini` segmenter ignores skip_ai_handoff).\n\nDETECTOR RUNNER selection (trajectory hybrids only; trajectory_hybrid.py:65 _resolve_runner). Precedence: RALLY_DETECTOR_IMPL env var > indexing.detector_impl (config default 'auto'):\n - 'auto' (default) -> the real WSL runner via subclass _build_runner: wasb_hybrid -> WasbRunner (WSL2 + conda env 'wasb' + pretrained WASB weights, GPU); tracknet -> TrackNet via WSL. NEEDS a GPU + WSL2 + conda env (config indexing.wasb/tracknet blocks). This is the production path NOT available on the CPU droplet.\n - 'native' / 'native_wasb' -> NativeWasbRunner (native_wasb_runner.py) via _build_native_runner ? Linux-native, no WSL/conda-activate, GPU box (M3.5, the NEXT build). ONLY wasb/fusion(substrate=wasb) support native (trajectory_hybrid.py:60 base refuses with a clear error; tracknet/yolo native -> NotImplementedError).\n - 'stub' -> StubDetectorRunner (stub_runner.py): deterministic CPU MOCK, no GPU/WSL ? makes the whole hybrid pipeline runnable/regression-testable on a CPU box/CI (\"mock a GPU with a CPU\"). This is what the decoupled CPU-droplet e2e used (RALLY_DETECTOR_IMPL=stub + skip_ai_handoff to also drop Gemini).\nSo the only GPU-free, key-free way to exercise a hybrid end-to-end is detector_impl=stub + skip_ai_handoff=true. gemini needs the key (no GPU). motion is the only fully-local, no-key, no-GPU REAL segmenter.",
    "ui_drive_gaps": [
      "EXISTS: a static SPA (backend/api/static/index.html + app.js + style.css) already drives the CORE loop ? process by server-local path (POST /api/process + 3s poll of /api/jobs/{id}), render validation results, render rally cards, filter via /api/query, select segments, POST /api/compile, and build the /api/highlights download link. So index->reel is covered for an already-on-server file.",
      "GAP ? browser upload not wired: the chunked upload endpoints (/api/upload/init|part|complete|abort) EXIST and work, but app.js never calls them; the UI only takes a typed server-local path string. A real UI needs a file-picker that chunks <=max_part_mb, drives init/part/complete, then feeds the returned absolute path into /api/process. (init exposes max_part_mb to size the chunks.)",
      "GAP ? no library/list endpoints: there is NO GET /api/videos and NO GET /api/jobs (list). A UI can only re-find a video if it kept the video_id in memory; on reload everything is lost. Need list-all-videos (with status) and list-all-jobs (history) endpoints + a backing db query ? currently only get_job(id)/get_video(id) single-row lookups exist.",
      "GAP ? segmenter/config not selectable in UI: /api/process accepts a `segmenter` field and GET /api/config exposes the registry/default, but app.js hardcodes only {video_path, player_pool} and never lets the user pick gemini vs wasb_hybrid vs motion, toggle skip_ai_handoff, or see which detector_impl/GPU/key the box supports. A full UI needs to surface available segmenters (from /api/config or list_available) + their requirements.",
      "GAP ? query response field mismatch / no 'list all segments' refresh: app.js reads data.rallies on /api/query (line 344) but the API returns {play_segments}; and the initial card list comes only from the process result, so there is no way to re-load a previously indexed video's segments without re-processing (no GET segments-by-video endpoint).",
      "GAP ? progress is coarse + poll-only: /api/jobs returns a single free-text `progress` stage string (hashing/validating/segmenting/finalizing); there is no percentage, no per-stage ETA, no streaming (SSE/WebSocket). A real UI shows just a spinner with the stage text. Also no cancel-job endpoint.",
      "GAP ? no auth / multi-user / per-user scoping: FastAPI has NO auth and CORS allow_origins='*' (main.py:42-46); upload_dir + output_dir are global (per-user partitioning is explicitly deferred to PR-3). A hosted UI can't isolate tenants today.",

```
      "GAP ? reels not discoverable: /api/highlights/{video_id}/{filename} streams a reel only
if you already know the exact output_filename you compiled with; there is no list-compiled-
reels-for-video endpoint, so a UI can't show a gallery of previously made reels after reload.",
      "GAP ? single-box concurrency ceiling: worker is max_workers=1 in-process, so the UI must
communicate 'one job at a time' (a 2nd submit queues behind the 1st); no progress/queue-position
is exposed, and a server restart fails in-flight jobs (UI should surface 'failed: interrupted by
server restart').",
      "GAP ? decoupled storage/worker path is invisible to the UI: the StorageBackend + `python
-m backend.worker` path (S3/GCS/DO Spaces) is entirely separate from the FastAPI; the UI drives
only the local-filesystem FastAPI loop. Driving the decoupled GPU-worker loop from a UI would
need new orchestration/endpoints that don't exist yet."
    ],
    "file_refs": [
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\main.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\api\\uploads.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\api\\job_worker.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\config.json",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\config\\models.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\pipeline\\segmenters\\base.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\pipeline\\segmenters\\__init__.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\pipeline\\segmenters\\trajectory_hybrid.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\pipeline\\segmenters\\wasb_hybrid.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\pipeline\\segmenters\\gemini.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\infrastructure\\database.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\api\\static\\app.js",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\api\\static\\index.html",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\pipeline\\detectors\\stub_runner.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\pipeline\\detectors\\native_wasb_runner.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\storage\\"
    ]
  },
  "hosting": {
    "exposure_auth": "INTENDED MODEL = single-tenant, private/invite-only, NO real auth shipped
yet. The engine FastAPI has ZERO auth and a wide-open CORS: backend/main.py:42-43
`allow_origins=[\"*\"]`, `allow_credentials=False` (only one \"auth\" hit in main.py, in a
comment at :39 ? no Depends/HTTPBearer/api_key guard exists). The dev SPA proxy also runs with
no auth (khelsutra web/README.md:2 \"no auth locally (DEV_USER_EMAIL)\").\n\nThe DESIGNED (not-
yet-built) exposure model is in docs/HOSTING_PLAN.md (status \"PROPOSAL ? awaiting owner
ratification\"): Alpha = Cloudflare edge (free) ? DNS/TLS + **Cloudflare Access email-OTP
allowlist** in front of BOTH app.khelsutra.guru (Pages SPA) and api.khelsutra.guru
(HOSTING_PLAN.md:26-31); a **Cloudflare Tunnel** (`cloudflared`, outbound-only, \"No inbound
ports opened ? Home IP never exposed\" :36) reaches FastAPI :8000 on the box. Auth = backend
trusts the `Cf-Access-Authenticated-User-Email` header / verify the Access JWT for per-user
scoping (HOSTING_PLAN.md:37-38). 100 MB edge body cap ? ?90 MB chunked upload (:39). Go-live
checklist (:102-108) DEMANDS the auth/exposure hardening that does NOT yet exist: Access
allowlist ON, CORS narrowed to https://app.khelsutra.guru, rate limit + per-user upload quota,
`security.allowed_video_root`, rotate the Gemini key. Beta graduates auth to Firebase Auth + GCS
direct upload (HOSTING_PLAN.md:46-67). PLATFORM_ARCHITECTURE.md:62 confirms \"API ? no auth\" is
a known gap; auth is P1 (build-vs-buy Auth0/Clerk/Supabase, §8.3). Khelsutra marketing site is
\"private/invite-only\" (KHELSUTRA_PRODUCT_OVERVIEW.md:41, :57). NET: a publicly-reachable box
today is unauthenticated; the only intended gate is Cloudflare Access (edge), not anything in
the app.",
    "locality_model": "docs/VIDEO_LOCALITY_MODEL.md (status \"THINKING DOC\", no greenlight).
Central thesis (§1, :30): \"timestamps are the product; the video is dead weight\" ? the
customer receives {rally intervals, report, reel} (kilobytes); only the highlight compiler needs
the original full-res bytes, and that is pure ffmpeg that can run wherever the original lives.
```

TODAY (L3, §0 :10-18): the chunked-upload flow assembles the full file under `security.upload_dir` (default output/uploads), registers by MD5, and **nothing ever deletes it** (retention = acknowledged gap, OQ1); the Gemini path also ships ?1280px re-encoded chunks to Google's Files API. Current cap = `security.max_upload_mb` default **4096 MB** (§0 :21), so a 10 GB upload is impossible today.\n\nThe \"locality ladder\" (§2 table :53-60): L3 full upload (built, capped, wrong for big files) ? **L2 proxy upload** (timeline-identical ?1080p re-encode, ~1-2.5 GB, the recommended alpha default §7 :140) ? L1 chunks-only / L1-direct (chunks straight to Gemini) ? **L0 fully local** (nothing leaves; the owned on-device model ? the desktop-app north star) ? L4 sneakernet (a hard drive; how the n=3 pilot runs today). Privacy framing: \"strictly no bytes leave only at L0\" and even L0 leaks ?1280px chunks to Google while Gemini is the brain ? must be in consent language (OQ3 :129). Storing the proxy not the original = ~10× less disk + smaller privacy surface (§3 :71-72), and the review UI prefers streaming a 1080p proxy (OQ6 :132). For a UI streaming source: L2 streams the proxy (good); L1 has only chunks ? review degrades ? this argues L2 as the default playback/clip source. Reel delivery undecided (OQ7 :133): server-compiled proxy reel (shareable 1080p) vs intervals-back + client cuts full-res. Consent/marketing (KHELSUTRA_PRODUCT_OVERVIEW.md:101-103): \"handled securely on our infrastructure\", never published, minors' footage never used to improve the tool, training opt-in only.",

"droplet_layout": "ONE shared CPU droplet 168.144.126.176 (DigitalOcean, Ubuntu 24.04, 2 vCPU / 3.8 GB, **NO GPU** ? SETUP_NEW_MACHINE.md:7) hosts BOTH:\n1) **The rally engine** at `~/rally` (git clone or scp'd archive; SETUP_NEW_MACHINE.md §3 :67-77). FastAPI on **port 8000** (HOSTING_PLAN.md:31; khelsutra web/README.md:2 dev proxy ? http://localhost:8000). Bootstrap = deploy/digitalocean/bootstrap.sh (mirrors CI: ffmpeg + libgl1 + .venv + CPU-or-CUDA torch + `pip install -e .[dev]`; verified torch 2.12.1+cpu, 922 passed/11 skipped). Storage layer = backend/storage/ (base/local/s3/gcs/gdrive/ref); worker = `python -m backend.worker {infer|train}` (backend/worker/__main__.py), URI-driven. Verified e2e 2026-06-20 reading/writing real DO Spaces (sgp1), GCS emulator, MinIO ? with the GPU MOCKED (`RALLY_DETECTOR_IMPL=stub`, `indexing.detector_impl=stub`, `skip_ai_handoff=true`; SETUP_NEW_MACHINE.md §6a :120-143).\n2) **The khelsutra marketing/request-access site** at `/srv/khelsutra-site` (isolated dir; PORTABILITY.md:32), a dependency-free Node `site/server.js` (node:http + node:sqlite) on **port 8787**, under systemd unit `khelsutra-site`, live at http://168.144.126.176:8787 (khelsutra/NEXT_STEPS.md:11-12; site/scripts/droplet-smoke.sh defaults REMOTE_DIR=/srv/khelsutra-site PORT=8787). Same code also deploys to Cloudflare Workers+D1 (PORTABILITY.md two-hosts table); the VM is the \"if Cloudflare acts out\" fallback.\n\nGPU box = a SEPARATE, NOT-YET-BUILT machine (cattle): a DO **GPU Droplet**, RTX 4000 Ada 20 GB ~$0.76/hr, in a GPU region (tor1/nyc2/atl1, NOT blr1 ? you cannot add a GPU to a CPU droplet; SETUP_NEW_MACHINE.md:43-47). Port procedure §9 :226-256: provision ? code to ~/rally ? `mount_scratch.sh` (formats+mounts the 2nd raw NVMe at /scratch, ephemeral, NOT in fstab; `export TMPDIR=/scratch`) ? `bootstrap.sh --gpu` (CUDA 12.4 torch) ? `gpu_setup.sh` (a SEPARATE miniconda env \"wasb\": py3.8 + torch 1.11.0+cu113, clones nttcom/WASB-SBDT, needs wasb_badminton_best.pth.tar) ? the \"M3.5\" native-WASB enabler. The real Linux-native DetectorRunner (NativeWasbRunner) is the NEXT build, not yet landed.\n\nSECRETS (the only non-reproducible items, never in repo): GEMINI_API_KEY at /root/.keys/GEMINI_API_KEY (chmod 600) and bucket creds in ~/.aws/credentials [default] (chmod 600) (SETUP_NEW_MACHINE.md:238-244). Bucket layout: `videos/ labels/ manifest.json weights/ outputs/<video_id>/` (:210). Teardown = `doctl compute droplet delete` (cattle).",

"desktop_vision": "YES ? a \"locally installed\" cross-platform desktop app IS already envisioned, in docs/DESKTOP_APP_PLAN.md (status \"DRAFT ? owner-gated; nothing built; the entire build is [design]\", :3-7). Pitch (§0 :11-12): \"Plug in your card, get your highlights\" ? package the mature local pipeline (WASB shuttle detector + windowing + ffmpeg stitch, already runnable offline via tools/generate_highlights.py --skip-ai-handoff) for non-technical recreational players; plug in USB/SD ? one click ? get back only the reels, 100% on-device, NO upload, NO copy of the 4K originals. It explicitly = the `local`/`LocalDesktop` ComputeBackend of PLATFORM_ARCHITECTURE.md §3.2 made into a product, and realizes the **L0 tier** of VIDEO_LOCALITY_MODEL (§4.3 :101-102, and DESKTOP_APP_PLAN header :6).\n\nSTACK (§3.4 :61-64, §4.4 :104-105): app shell = lean \"CLI + installer MVP wrapping the existing `indexer` console entry / generate_highlights.py ? graduate to a **Tauri** GUI (Rust + system webview, ~3-10 MB, MIT/Apache), with the **Python/FastAPI backend bundled as a localhost sidecar via PyInstaller** (PEP 621 pyproject + `indexer` entry point already shipped, #167). Electron is the fallback. Detector = a NEW **NativeWasbRunner** (DetectorRunner subclass, NO WSL ? drops the WSL2 dependency, D3 :37) lifting backend/pipeline/detectors/wasb_infer.py to native torch/onnxruntime per-OS, keeping the identical Frame,Visibility,X,Y CSV; windowing + VideoCompiler stitch reused verbatim. Licensing (D4 :38): MIT/Apache/BSD only, ffmpeg as an **LGPL build** (openh264/hardware encoders, no GPL libx264). \n\nCRUX/gate (§0 :14, §5, P0): compute on a typical no-GPU enthusiast laptop is the make-or-break unknown ? WASB is ~3.4× real-time on the

```
owner's RTX 2070S [measured] but CPU/Apple-MPS throughput is UNVERIFIED; P0 is a feasibility
spike that must measure it before any app code, and everything past P0 is gated. Relation to
browser appliance: it's the privacy-max end of the SAME spectrum as the hosted SPA ? same
pipeline, control/data-plane \"degenerates to one machine\" (§4.3 :102); Gemini stays an
optional opt-in toggle, never required.",
    "single_tenant_assumptions": [
      "NO app-layer auth exists: backend/main.py FastAPI ships open CORS allow_origins=['*'] and
zero auth dependency ? a publicly-reachable box is wide open unless something external (the
planned Cloudflare Access edge allowlist) gates it; HOSTING_PLAN go-live checklist (:102-108)
lists CORS-narrowing + Access + rate-limit + per-user quota as REQUIRED-but-not-yet-done before
exposure.",
      "The intended gate is edge-only and email-allowlist, not real multi-tenant identity:
Cloudflare Access email-OTP for ?50 alpha testers, backend trusting Cf-Access-Authenticated-
User-Email (HOSTING_PLAN.md:37-38). Multi-tenant identity/RLS/per-tenant isolation is P1 and
unbuilt (PLATFORM_ARCHITECTURE.md §4.2, §6).",
      "Cloudflare Tunnel keeps the home/box IP unexposed and opens NO inbound ports (cloudflared
dials out; HOSTING_PLAN.md:36) ? the single box is reachable only via the edge, never directly,
in the intended design (the raw 168.144.126.176:8000/:8787 exposure today is a testbed, not the
production posture).",
      "Retention is an UNSOLVED guardrail gap: uploaded originals under output/uploads are never
deleted (VIDEO_LOCALITY_MODEL §0 :10-18, OQ1) ? a real privacy/liability hole for a single
publicly-reachable box holding many users' video; default to keeping only proxies + intervals.",
      "Minors are excluded by default from the training pool, and per-user/contributed-footage
training requires a SEPARATE explicit opt-in (CLAUDE.md guardrails; PLATFORM_ARCHITECTURE.md §7
:516-526; KHELSUTRA_PRODUCT_OVERVIEW.md:98-103) ? never train on what merely passes through a
tenant's bucket.",
      "Paid LLMs (Gemini/OpenAI/Anthropic) are inference/serving/fallback ONLY, never a training
source; even 'local' modes leak ?1280px chunks to Google's Files API on the Gemini path, which
MUST be stated in consent language (VIDEO_LOCALITY_MODEL OQ3; PLATFORM_ARCHITECTURE §5A teacher
constraint).",
      "Per-video cost ceiling needed so one box can't be surprise-billed: cap GPU/Gemini spend
per video, ai_max_workers=1, serial uploads, bounded retry (VIDEO_LOCALITY_MODEL §4 :84-103).",
      "Secrets live only on the box (GEMINI_API_KEY in /root/.keys chmod 600, bucket creds in
~/.aws/credentials chmod 600) ? never in repo/PR/config.json; rotate the previously-chat-shared
Gemini key (SETUP_NEW_MACHINE.md:238-244; HOSTING_PLAN.md:106).",
      "The droplet is a SHARED CPU-only testbed (MinIO 9000/9001, GCS emulator 9023, no GPU);
real GPU WASB is a separate cattle GPU droplet in a non-blr1 region ? do not assume GPU or treat
shared services as production."
    ],
    "file_refs": [
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\HOSTING_PLAN.md",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\PLATFORM_ARCHITECTURE.md",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\VIDEO_LOCALITY_MODEL.md",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\DESKTOP_APP_PLAN.md",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\SETUP_NEW_MACHINE.md",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\docs\\KHELSUTRA_PRODUCT_OVERVIEW.md",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\deploy\\digitalocean\\README.md",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\deploy\\digitalocean\\bootstrap.sh",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\deploy\\digitalocean\\gpu_setup.sh",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\deploy\\digitalocean\\mount_scratch.sh",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\main.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\worker\\__main__.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\storage\\base.py",
      "C:\\Users\\avidu\\Projects\\khelsutra\\NEXT_STEPS.md",
      "C:\\Users\\avidu\\Projects\\khelsutra\\docs\\PORTABILITY.md",
      "C:\\Users\\avidu\\Projects\\khelsutra\\web\\README.md",
      "C:\\Users\\avidu\\Projects\\khelsutra\\site\\server.js",
      "C:\\Users\\avidu\\Projects\\khelsutra\\site\\scripts\\droplet-smoke.sh"
    ]
  },
```

```
  "uiRepo": {
    "site_ui_surface": "khelsutra repo (cwd C:\\Users\\avidu\\Projects\\khelsutra, private, all-rights-reserved, master) has THREE UI surfaces, two live + one scaffold:\n\n1) site/ ? the PUBLIC marketing landing page. SHIPPED + LIVE on khelsutra.guru. Runtime-portable static-asset + Worker app: `site/src/index.js` (Cloudflare Worker `fetch`) and `site/server.js` (node:http twin) both call the same `handleRequest(request, store)` over a `store` adapter (`site/src/store.js`: `d1Store` | `sqliteStore`) [PORTABILITY.md:8-23]. Public pages: `site/public/index.html` (landing) + `site/public/capture.html` (recording checklist) + `site/public/og.png`. It hosts the segmented Request-Access lead form ? D1 (`site/schema.sql`, `access_requests`) [REQUEST_ACCESS_FORM.md:42-66]. README.md:13: \"site/ ? public marketing landing page (static; served at khelsutra.guru)\". Auto-deploys: merge to master ? Cloudflare Workers Builds (root site/) [NEXT_STEPS.md:9-12]. Also runs as a Node+SQLite twin live on the DO droplet at http://168.144.126.176:8787 (systemd khelsutra-site) ? the SAME droplet the engine lives on.\n\n2) web/ ? the ALPHA React SPA. SCAFFOLD-ONLY today: web/ contains just README.md (no code yet). README.md:14: \"web/ ? React SPA (alpha; Vite + React + TS + Tailwind; Cloudflare Pages build root)\"; web/README.md:1-2: \"SPA scaffold lands here at execution-plan milestone M1 (Vite + React + TS + Tailwind). Dev: talks to the engine at http://localhost:8000 via Vite proxy; no auth locally (DEV_USER_EMAIL).\" This is where the operator/per-rally UI belongs.\n\n3) control-plane/ ? (beta) Cloud Run FastAPI control plane; README.md:16 ? not yet created.\n\nHow the appliance \"operator console\" relates to the public marketing site: they are deliberately SEPARATE surfaces with a gating boundary baked into the planned topology. The public marketing page (site/, khelsutra.guru, Cloudflare Pages/Worker, $0) is UNAUTHENTICATED ? anyone can read it and submit the lead form. The operator console / appliance UI is a GATED APP: per HOSTING_PLAN.md:24-44 the alpha topology splits app.khelsutra.guru (Cloudflare Pages ? the web/ SPA, behind Cloudflare Access email-OTP allowlist) from api.khelsutra.guru (Cloudflare Tunnel ? FastAPI :8000 on the box). So the operator console is the web/ SPA sitting BEHIND Cloudflare Access, talking to the engine API ? NOT a public page. UI_PROPOSAL.md:33-37 confirms: SPA on static hosting, API on the box via tunnel, auth = Cloudflare Access email-allowlist (zero auth code; backend reads the authenticated-email header). Grounding caveat to record honestly: the engine's FastAPI today has NO auth (CORS '*'), so the Access/tunnel gate is the ONLY thing standing between the public internet and the appliance ? the console must assume the gate, not the app, is the tenancy boundary in alpha. The marketing site sells/funnels (Request-Access form captures GPU + storage answers = the BYO-compute/BYO-storage demand signal); the gated console operates the decoupled engine (the just-shipped storage + worker decoupling). A reasonable IA: keep the console as one or more authed screens in the SAME web/ SPA (Cloudflare Pages app root), distinct from site/ which stays the public Worker ? they already auto-deploy on separate roots (site/ Worker vs web/ Pages) so docs/web/ changes don't touch the live site/ Worker [PER_RALLY_SELECTION.md:108].",
    "per_rally_relation": "The just-merged docs/PER_RALLY_SELECTION.md (PR #11, status IN PROGRESS, owner-approved 2026-06-20) is the FIRST screen of the broader operator console ? it is the \"pick rallies ? reel\" workhorse, not the whole console. Concretely:\n\n- PER_RALLY_SELECTION.md is the RallyPicker: a hybrid selectable rally grid + deterministic query bar that rides EXISTING engine endpoints with ZERO new engine code for its MVP ? POST /api/query ? select segment_ids ? POST /api/compile ? GET /api/highlights [PER_RALLY_SELECTION.md:12, §4.1, D6]. It maps to alpha screen A5 Highlights (engine UI_PROPOSAL.md:47) and milestone M4 (UI_ALPHA_EXECUTION_PLAN.md), with its correction-loop extension realizing A4 Rally Review (UI_PROPOSAL.md:46).\n\n- An \"operator console\" for the DECOUPLED engine is a SUPERSET. PER_RALLY_SELECTION covers the curation-of-results step only; the console additionally needs the screens around it that UI_PROPOSAL.md:41-48 already enumerates ? A1 Matches (videos + verdict chips), A2 Upload & Process, A3 Job progress, A6 Report ? PLUS the genuinely new decoupled-era operations that PER_RALLY_SELECTION explicitly excludes: kicking off/monitoring worker infer/train jobs against a bucket, choosing a StorageBackend/bucket, per-video/per-run cost attribution, and weights/model versions. PER_RALLY_SELECTION is job-tethered by design (D10: carries result.video_id, no GET /api/videos) and deliberately punts multi-video home (P8, owner-gated), per-rally preview/timeline (P9), and the correction loop (P10) ? those punted milestones ARE the rest of the operator console.\n\n- So the relation is: per-rally selection = ONE screen (the reel-builder) of the console; the console wraps it with match list, upload, job-launch/monitor (the new decoupled worker verbs), report, and cost/model-version surfaces. The appliance tracked-doc should declare itself a peer-of PER_RALLY_SELECTION.md (and of REQUEST_ACCESS_FORM.md), reuse its verified engine-contract anchors (§3, §10) verbatim rather than re-deriving them, and adopt its honesty discipline (never surface motion-only server/receiver/events; never render a confidence meter from the Gemini string). It must NOT duplicate the RallyPicker design ? it should point to it as the embedded curation screen and own the job-orchestration/storage/cost surfaces PER_RALLY_SELECTION does not.",
    "doc_template": "Follow the engine repo's docs/PROJECT_DOC_TEMPLATE.md
```

(C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\PROJECT_DOC_TEMPLATE.md) ? the SAME template both existing khelsutra tracked docs mirror (PER_RALLY_SELECTION.md and REQUEST_ACCESS_FORM.md self-describe as \"mirrors the engine repo's PROJECT_DOC_TEMPLATE discipline\"). The exact structure to fill in:\n\nSTATUS HEADER (blockquote, 5 lines) ? > **Status:** `DRAFT` ? `<owner-gated? | in progress>` · **Owner:** Avi · **Created:** 2026-06-20 · **Last updated:** 2026-06-20 ; > **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to docs/archives/ when DONE) ; > **Tracking anchors:** §7 progress tracker is the source of truth; indexed in docs/README.md; pointer in NEXT_STEPS.md + memory current-state once work starts ; > **Relation to existing docs:** `<peer-of PER_RALLY_SELECTION.md / REQUEST_ACCESS_FORM.md; extends UI_PROPOSAL.md alpha screens>` ; > **Honesty note:** mark each claim [verified] / [researched] / [design]; not legal/financial advice.\n\nBODY SECTIONS (delete OPTIONAL ones that don't apply):\n- §0 TL;DR (2?5 sentences: goal, approach, single most important caveat)\n- §1 Problem & goal (why now; concrete user outcome; links to read to get current)\n- §2 Decisions locked (table: # | Decision | Source / date | Implication) ? capture answered questions so they aren't relitigated\n- §3 Foundation ? research / prior art (OPTIONAL; include for engine-fact-heavy work, which this is ? use it for the verified engine/worker/storage contract, like PER_RALLY_SELECTION §3 did)\n- §4 Design / architecture (the plan + a REUSE MAP naming existing modules so new-vs-reused is explicit)\n- §5 Threat model / risk table (OPTIONAL ? relevant here: no engine auth, CORS '*', shared testbed droplet)\n- §6 Honest limits ? what this does NOT do (false-confidence guardrail)\n- §7 Deliverables & progress tracker ? SOURCE OF TRUTH (table: ID | Deliverable | Depends on | Gated? | Status | PR; legend ? Todo · ? In progress · ? Done · ? Blocked/gated; ONE small PR per row, branched from origin/master, NO stacking)\n- §8 Open questions ? owner / external (split blocking-gates from nice-to-knows; name who decides)\n- §9 Definition of done (explicit acceptance checklist; flip Status?DONE + archive when all ticked)\n- §10 References (Internal code anchors [verified] with paths; internal docs; external)\n- Changelog (dated entries at the bottom)\n\nHouse-voice rules from the template comment block (lines 9-16): §7 is the source of truth (one row per independently-shippable PR); keep it HONEST ? tag every claim [verified]/[researched]/[design], reflect real shipped state never aspirational; POINT to detail (code anchors), don't duplicate it; delete OPTIONAL sections that don't apply; move to docs/archives/ once Status=DONE (terminal-state only). After creating it, index it in khelsutra docs/README.md \"Tracked work (this repo)\" and add a pointer in khelsutra NEXT_STEPS.md (the same wiring PER_RALLY_SELECTION got).",

    "reuse_opportunities": [

      "Decoupled engine seams (just-shipped, master) ? the appliance's whole reason to exist: backend/storage/ StorageBackend ABC (base.py: fetch_to_local/put/exists/stat/list/delete + signed_url) with URI-driven local + s3 (AWS/DO Spaces/R2/MinIO) + gcs backends and lazy SDK import (__init__.py _backend_class), and the worker dispatcher backend/worker/__main__.py `python -m backend.worker {infer|train}` (pull from storage ? SAME pipeline building blocks ? push). The console drives THESE ? do not re-plumb storage or fork the pipeline; the worker is 'a thin orchestrator' over backend.main's blocks.",

      "RallyPicker design + its verified engine-contract anchors ? reuse docs/PER_RALLY_SELECTION.md §3/§4/§10 (POST /api/query ? segment_ids ? POST /api/compile ? GET /api/highlights; GET /api/config min_shot_count; GET /api/jobs/{id} result.video_id) verbatim instead of re-deriving; embed RallyPicker as the console's curation screen rather than redesigning it.",

      "Host-portability adapter pattern from PORTABILITY.md ? runtime-agnostic handleRequest(request, store) + d1Store/sqliteStore injection (site/src/store.js); the same 'logic-in-handler, host in adapter' shape is exactly how the StorageBackend registry already works in the engine, so the console can speak one URI contract across local/S3/GCS without host-specific branches.",

      "Parity + CI discipline ? site/scripts/parity-check.sh + .github/workflows/parity.yml (scheduled live byte-for-byte drift check between the Cloudflare host and the droplet twin) and test.yml (npm test green on every PR, blocks merge); reuse this dual-host-parity + green-CI-gates-merge pattern for any console screen that has both a live droplet and a Cloudflare path. node --test runs BOTH host paths (PORTABILITY.md:25-30).",

      "SPA scaffold + dev seam ? web/ (Vite + React + TS + Tailwind, Cloudflare Pages build root) with Vite proxy to engine :8000 and DEV_USER_EMAIL for no-auth local dev (web/README.md); the console screens land in this same scaffold (planned exec-plan M1) ? reuse the typed API client / useRallySelection() hook seams PER_RALLY_SELECTION P1-P2 introduce.",

      "Tracked-doc + workflow conventions ? docs/PROJECT_DOC_TEMPLATE.md structure, the docs/README.md index + NEXT_STEPS.md pointer wiring, REQUEST_ACCESS_FORM.md's storage/GPU questions as the demand signal that justifies the appliance, and the one-PR-per-row-off-origin/master no-stacking discipline.",

      "Gated-topology blueprint already written ? HOSTING_PLAN.md alpha topology (Cloudflare

```
Access email-OTP in front of app.khelsutra.guru Pages + api.khelsutra.guru Tunnel ? FastAPI on
the box, /api/health offline banner) is the ready-made gating model for the operator console;
reuse it rather than inventing auth, and note the engine-has-no-auth caveat as a risk."
    ],
    "file_refs": [
      "C:\\Users\\avidu\\Projects\\khelsutra\\README.md:11-16",
      "C:\\Users\\avidu\\Projects\\khelsutra\\web\\README.md:1-2",
      "C:\\Users\\avidu\\Projects\\khelsutra\\docs\\PER_RALLY_SELECTION.md:6,12,108,124",
      "C:\\Users\\avidu\\Projects\\khelsutra\\docs\\PORTABILITY.md:8-30",
      "C:\\Users\\avidu\\Projects\\khelsutra\\docs\\README.md:3-13",
      "C:\\Users\\avidu\\Projects\\khelsutra\\docs\\REQUEST_ACCESS_FORM.md:5-9,42-66,88-94",
      "C:\\Users\\avidu\\Projects\\khelsutra\\NEXT_STEPS.md:9-26,37-41",
      "C:\\Users\\avidu\\Projects\\khelsutra\\.github\\workflows\\parity.yml",
      "C:\\Users\\avidu\\Projects\\khelsutra\\.github\\workflows\\test.yml",
      "C:\\Users\\avidu\\Projects\\khelsutra\\site\\src\\store.js",
      "C:\\Users\\avidu\\Projects\\khelsutra\\site\\src\\index.js",
      "C:\\Users\\avidu\\Projects\\khelsutra\\site\\server.js",
      "C:\\Users\\avidu\\Projects\\khelsutra\\site\\scripts\\parity-check.sh",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\PROJECT_DOC_TEMPLATE.md",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\UI_PROPOSAL.md:41-60",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\HOSTING_PLAN.md:22-44",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\docs\\DECOUPLED_COMPUTE\\README.md:18-29",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\docs\\DECOUPLED_COMPUTE\\HANDOFF.md:5-30,50-64",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\storage\\base.py:14-49",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\storage\\__init__.py:28-45",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\worker\\__main__.py:1-16"
    ]
  }
}

Return a concrete, buildable design under this lens. mvp_scope = the smallest slice that drives
the full CUJ on a single box (CPU droplet acceptable via gemini/stub).
```

2. assistant (2026-06-20T09:03:00.514Z)

I'll design this appliance. Let me first ground myself in the key files to make sure my design reconciles with the actual built state.

3. user (2026-06-20T09:03:05.270Z)

```
fatal: Cannot fast-forward to multiple branches.
---BRANCH---
feat/native-wasb-runner
---HANDOFF EXISTS?---
01-EXISTING-PLANS-AND-GAP.md
02-COMPUTE-MAP-AND-CONTRACT.md
03-PLATFORM-OPTIONS.md
04-BUILD-AND-TEST-PLAN.md
05-COST-ATTRIBUTION.md
06-RISKS-AND-OPEN-QUESTIONS.md
07-COMPUTE-ABSTRACTION.md
HANDOFF.md
README.md
```

4. user (2026-06-20T09:03:05.280Z)

```
1       // Badminton Highlight & Rally Indexer JS Client
2
3       document.addEventListener("DOMContentLoaded", () => {
```

```
        // --- APP STATE ---
        let currentVideoId = null;
        let indexedRallies = [];
        let selectedRallyIds = new Set();
        let playerPool = ["Avi", "Suresh", "Ramesh", "Deepak"];

        // --- UI ELEMENTS ---
        const videoPathInput = document.getElementById("video-path-input");
        const playerPoolInput = document.getElementById("player-pool-input");
        const btnProcess = document.getElementById("btn-process-video");

        const validationList = document.getElementById("validation-checker-list");
        const ralliesResultsList = document.getElementById("rallies-results-list");
        const rallyCountLabel = document.getElementById("rally-count-label");

        // Query Filters
        const queryServer = document.getElementById("query-server");
        const queryReceiver = document.getElementById("query-receiver");
        const queryDuration = document.getElementById("query-duration");
        const queryEvent = document.getElementById("query-event");
        const btnQuery = document.getElementById("btn-run-query");

        // Compiler Bar
        const compileBoard = document.getElementById("highlight-compile-board");
        const compileSelectedCount = document.getElementById("compile-selected-count");
        const highlightOutputName = document.getElementById("highlight-output-name");
        const btnCompile = document.getElementById("btn-compile-highlights");

        // Loading Overlay
        const loadingOverlay = document.getElementById("loading-overlay");
        const loadingTitle = document.getElementById("loading-title");
        const loadingSubtitle = document.getElementById("loading-subtitle");

        // --- HELPERS ---
        const showLoader = (title, subtitle) => {
            loadingTitle.textContent = title;
            loadingSubtitle.textContent = subtitle;
            loadingOverlay.style.display = "flex";
        };

        const hideLoader = () => {
            loadingOverlay.style.display = "none";
        };

        const populatePlayerSelects = (players) => {
            // Clear except first
            queryServer.innerHTML = '<option value="">Any Server</option>';
            queryReceiver.innerHTML = '<option value="">Any Receiver</option>';

            players.forEach(player => {
                const opt1 = document.createElement("option");
                opt1.value = player;
                opt1.textContent = player;
                queryServer.appendChild(opt1);

                const opt2 = document.createElement("option");
                opt2.value = player;
                opt2.textContent = player;
                queryReceiver.appendChild(opt2);
            });
        };

        // Initialize player selects at start
        populatePlayerSelects(playerPool);

```

```
        // --- 1. RUN VIDEO PROCESSING & VALIDATIONS ---
        btnProcess.addEventListener("click", async () => {
            const videoPath = videoPathInput.value.trim();
            if (!videoPath) {
                alert("Please enter a valid local MP4 video file path.");
                return;
            }

            // Parse custom players
            const rawPlayers = playerPoolInput.value.trim();
            if (rawPlayers) {
                playerPool = rawPlayers.split(",").map(p => p.trim()).filter(p => p.length >
0);
                populatePlayerSelects(playerPool);
            }

            showLoader("Analyzing Video Stream", "Running structural diagnostics, video
focus checks, and relevance validators...");

            try {
                // Async job model (#16): POST returns 202 + job_id; poll /api/jobs/{id}.
                const response = await fetch("/api/process", {
                    method: "POST",
                    headers: { "Content-Type": "application/json" },
                    body: JSON.stringify({
                        video_path: videoPath,
                        player_pool: playerPool
                    })
                });

                if (!response.ok) {
                    const err = await response.json();
                    throw new Error(err.detail || "Server failed to process the request.");
                }

                const submitted = await response.json();
                const jobId = submitted.job_id;

                // Poll until the job reaches a terminal state (done | failed).
                let job;
                for (;;) {
                    const jr = await fetch(`/api/jobs/${jobId}`);
                    if (!jr.ok) {
                        const err = await jr.json();
                        throw new Error(err.detail || "Failed to poll job state.");
                    }
                    job = await jr.json();
                    if (job.status === "done" || job.status === "failed") break;
                    showLoader("Analyzing Video Stream",
                        `Job ${job.status}${job.progress ? " ? " + job.progress : ""}?`);
                    await new Promise(res => setTimeout(res, 3000));
                }

                if (job.status === "failed") {
                    throw new Error(job.error || "Processing job failed.");
                }

                const data = job.result || {};
                currentVideoId = data.video_id;

                // Render validation list
                renderValidationResults(data.results);

                if (data.status === "failed") {
                    hideLoader();
```

```
                    alert("Validation Failed! This video does not meet our quality or
structural criteria. Please review the checklist.");
                    renderEmptyRallies("Validation failed. Please load a valid badminton raw
video.");
                    compileBoard.style.display = "none";
                    return;
                }

                // Successfully processed (field is play_segments ? the old `rallies` read
was a bug)
                indexedRallies = data.play_segments || [];
                selectedRallyIds.clear();
                renderRallyCards(indexedRallies);

                hideLoader();
                alert(`Success! Successfully processed and indexed ${indexedRallies.length}
badminton rallies.`);

            } catch (error) {
                hideLoader();
                alert(`Error: ${error.message}`);
                renderEmptyRallies("Error processing file.");
            }
        });

        // --- 2. RENDER VALIDATION CHECKLIST ---
        const renderValidationResults = (results) => {
            validationList.innerHTML = "";

            results.forEach(res => {
                const div = document.createElement("div");
                div.className = `validation-item ${res.passed ? 'passed' : 'failed'}`;

                const icon = document.createElement("div");
                icon.className = "val-icon";
                icon.textContent = res.passed ? "?" : "?";

                const info = document.createElement("div");
                info.className = "val-info";

                const title = document.createElement("h4");
                title.textContent = formatValidatorName(res.validator_name);

                const desc = document.createElement("p");
                desc.textContent = res.message;

                info.appendChild(title);
                info.appendChild(desc);

                // If there are detailed metrics, render them
                if (res.details && Object.keys(res.details).length > 0) {
                    const detailsSpan = document.createElement("span");
                    detailsSpan.className = "val-details";
                    detailsSpan.textContent = formatDetails(res.details);
                    info.appendChild(detailsSpan);
                }

                div.appendChild(icon);
                div.appendChild(info);
                validationList.appendChild(div);
            });
        };

        const formatValidatorName = (name) => {
            return name.split("_").map(w => w.charAt(0).toUpperCase() + w.slice(1)).join("
```

```
");
        };

        const formatDetails = (details) => {
            return Object.entries(details)
                .map(([k, v]) => `${k}: ${v}`)
                .join(" | ");
        };

        // --- 3. RENDER RALLY TIMELINE CARDS ---
        const renderRallyCards = (rallies) => {
            ralliesResultsList.innerHTML = "";
            selectedRallyIds.clear();
            updateCompilerBar();

            rallyCountLabel.textContent = `${rallies.length} Rallies Found`;

            if (!rallies || rallies.length === 0) {
                renderEmptyRallies("No rallies found matching your filters.");
                return;
            }

            rallies.forEach(rally => {
                const card = document.createElement("div");
                card.className = `rally-card ${selectedRallyIds.has(rally.id) ? 'selected' :
''}`;
                card.setAttribute("id", `rally-card-${rally.id}`);

                const meta = document.createElement("div");
                meta.className = "rally-meta";

                const title = document.createElement("h3");
                title.textContent = `Rally #${rally.id} (Serve: ${rally.server} ? Receive:
${rally.receiver})`;

                const sub = document.createElement("p");
                sub.innerHTML = `
                    <span><strong>Start:</strong> ${formatTime(rally.start_time)}</span>
                    <span><strong>End:</strong> ${formatTime(rally.end_time)}</span>
                `;

                const tagsDiv = document.createElement("div");
                tagsDiv.className = "rally-tags";

                // Long rally tag
                if (rally.duration > 10) {
                    const longTag = document.createElement("span");
                    longTag.className = "rally-tag long";
                    longTag.textContent = "Long Rally";
                    tagsDiv.appendChild(longTag);
                }

                // Check events to see if we have smashes or fouls
                const events = rally.events || [];
                const hasSmash = events.some(e => e.type === "smash");
                const hasFoul = events.some(e => e.type === "foul");

                if (hasSmash) {
                    const smashTag = document.createElement("span");
                    smashTag.className = "rally-tag smash";
                    smashTag.textContent = "Contains Smash";
                    tagsDiv.appendChild(smashTag);
                }

                if (hasFoul) {
```

```
                    const foulTag = document.createElement("span");
                    foulTag.className = "rally-tag foul";
                    foulTag.textContent = "Ends in Foul";
                    tagsDiv.appendChild(foulTag);
                }

                meta.appendChild(title);
                meta.appendChild(sub);
                meta.appendChild(tagsDiv);

                const action = document.createElement("div");
                action.className = "rally-action";

                const dur = document.createElement("div");
                dur.className = "rally-dur";
                dur.textContent = `${rally.duration.toFixed(1)}s`;

                const chk = document.createElement("div");
                chk.className = "checkbox-custom";

                action.appendChild(dur);
                action.appendChild(chk);

                card.appendChild(meta);
                card.appendChild(action);

                // Card Toggle Selection Click Listener
                card.addEventListener("click", () => {
                    if (selectedRallyIds.has(rally.id)) {
                        selectedRallyIds.delete(rally.id);
                        card.classList.remove("selected");
                    } else {
                        selectedRallyIds.add(rally.id);
                        card.classList.add("selected");
                    }
                    updateCompilerBar();
                });

                ralliesResultsList.appendChild(card);
            });
        };

        const renderEmptyRallies = (msg) => {
            ralliesResultsList.innerHTML = `
                <div class="empty-state">
                    <svg width="48" height="48" fill="none" stroke="currentColor" stroke-
width="1.5" viewBox="0 0 24 24"><path stroke-linecap="round" stroke-linejoin="round" d="M3.75
5.25h16.5m-16.5 4.5h16.5m-16.5 4.5h16.5m-16.5 4.5h16.5"/></svg>
                    <h3>No Rallies Available</h3>
                    <p>${msg}</p>
                </div>
            `;
        };

        const formatTime = (seconds) => {
            const m = Math.floor(seconds / 60);
            const s = Math.floor(seconds % 60);
            const ms = Math.floor((seconds % 1) * 10);
            return `${m.toString().padStart(2, '0')}:${s.toString().padStart(2,
'0')}.${ms}`;
        };

        // --- 4. RALLY TIMELINE FILTER QUERY ---
        btnQuery.addEventListener("click", async () => {
            if (!currentVideoId) {
```

```
                alert("Please process a video first before running filters.");
                return;
            }

            const server = queryServer.value;
            const receiver = queryReceiver.value;
            const duration = parseFloat(queryDuration.value) || null;
            const event = queryEvent.value;

            showLoader("Filtering Database Index", "Fetching rallies matching exact
relational conditions...");

            try {
                const res = await fetch("/api/query", {
                    method: "POST",
                    headers: { "Content-Type": "application/json" },
                    body: JSON.stringify({
                        video_id: currentVideoId,
                        server: server || null,
                        receiver: receiver || null,
                        duration_min: duration || null,
                        event_type: event || null
                    })
                });

                if (!res.ok) throw new Error("Failed to filter index.");

                const data = await res.json();
                renderRallyCards(data.rallies || []);
            } catch (err) {
                alert(`Query failed: ${err.message}`);
            } finally {
                hideLoader();
            }
        });

        // --- 5. COMPILER BOARD MANAGEMENT ---
        const updateCompilerBar = () => {
            const count = selectedRallyIds.size;
            if (count > 0) {
                compileBoard.style.display = "flex";
                compileSelectedCount.textContent = `${count} Rally Segment${count > 1 ? 's'
: ''} Selected`;
            } else {
                compileBoard.style.display = "none";
            }
        };

        btnCompile.addEventListener("click", async () => {
            if (!currentVideoId || selectedRallyIds.size === 0) {
                alert("No rallies selected for compiling highlights.");
                return;
            }

            const outputName = highlightOutputName.value.trim() ||
"highlights_compiled.mp4";
            showLoader("Compiling Highlight Reels", "Triggering localized high-speed FFmpeg
stitching demuxers...");

            try {
                const res = await fetch("/api/compile", {
                    method: "POST",
                    headers: { "Content-Type": "application/json" },
                    body: JSON.stringify({
                        video_id: currentVideoId,
```

```
                        // API field is segment_ids (the old rally_ids never matched the
request model)
                        segment_ids: Array.from(selectedRallyIds),
                        output_filename: outputName
                    })
                });

                if (!res.ok) {
                    const err = await res.json();
                    throw new Error(err.detail || "Concatenation failed.");
                }

                await res.json();
                // B3 (PR-2): offer the reel via the streaming download endpoint instead of
                // alerting a server-local path the browser can't reach.
                const dlUrl =
`/api/highlights/${currentVideoId}/${encodeURIComponent(outputName)}`;
                let dl = document.getElementById("highlight-download-link");
                if (!dl) {
                    dl = document.createElement("a");
                    dl.id = "highlight-download-link";
                    dl.style.cssText = "display:block;margin-top:10px;font-weight:bold;";
                    compileBoard.appendChild(dl);
                }
                dl.href = dlUrl;
                dl.setAttribute("download", outputName);
                dl.textContent = `? Download ${outputName}`;
                alert(`Highlight compilation completed! Use the download link to save
${outputName}.`);

            } catch (error) {
                alert(`Stitching compilation failed: ${error.message}`);
            } finally {
                hideLoader();
            }
        });
    });

```

5. user (2026-06-20T09:03:05.280Z)

```
      """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.

          python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
              [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
          python -m backend.worker train ...        # Gen-0 train-from-bucket (follow-up)

      The URI scheme selects the storage backend via ``backend.storage`` (a bare path /
``local://``
      is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing
local runs
      are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
      ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini
key) ?
      the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).

      This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
CLI uses
      (validator registry ? segmenter ? report), never a forked pipeline.
      """
      from __future__ import annotations

      import argparse
      import csv
      import json
```

```
    import os
    import sys
    import tempfile
    from typing import Any, List

    from backend import storage
    from backend.config import load_config
    from backend.infrastructure.database import Database
    from backend.pipeline.segmenters.base import segmenter_registry
    from backend.pipeline.validators.base import registry as validator_registry
    from backend.reporting import emit_indexer_report
    from backend.results import Failure
    from backend.utils.hashing import compute_video_id


    def _coerce(v: str) -> Any:
        """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
str."""
        low = v.lower()
        if low in ("true", "false"):
            return low == "true"
        for cast in (int, float):
            try:
                return cast(v)
            except ValueError:
                pass
        return v


    def _apply_overrides(config: Any, sets: List[str]) -> None:
        """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
indexing.skip_ai_handoff=true)."""
        for kv in sets or []:
            key, sep, val = kv.partition("=")
            if not sep:
                raise ValueError(f"--set expects k=v, got {kv!r}")
            parts = key.split(".")
            obj = config
            for p in parts[:-1]:
                obj = getattr(obj, p)
            setattr(obj, parts[-1], _coerce(val))


    def _write_intervals_csv(segments: List[dict], path: str) -> None:
        """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
friendly
        artifact (same schema as the rally-annotator labels)."""
        with open(path, "w", newline="") as f:
            w = csv.writer(f)
            w.writerow(["start", "end", "shot_count", "ending_reason"])
            for s in segments:
                w.writerow([
                    f'{float(s.get("start_time", 0.0)):.3f}',
                    f'{float(s.get("end_time", 0.0)):.3f}',
                    s.get("shot_count", ""),
                    s.get("ending_reason", s.get("shot_count_confidence", "")),
                ])


    def _write_report_json(video_id: str, segments: List[dict], status: str, path: str) ->
None:
        with open(path, "w") as f:
            json.dump({
                "video_id": video_id,
                "status": status,
```

```
                "segment_count": len(segments),
                "segments": [{"start": float(s.get("start_time", 0.0)),
                              "end": float(s.get("end_time", 0.0))} for s in segments],
            }, f, indent=2)


    def cmd_infer(args: argparse.Namespace) -> int:
        config = load_config(args.config) if args.config else load_config()
        _apply_overrides(config, args.set)
        storage_cfg = config.storage.model_dump()

        scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
        os.makedirs(scratch, exist_ok=True)
        config.output.output_dir = scratch

        # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
        local_video = storage.fetch(args.video_uri, os.path.join(scratch, "in.mp4"),
config=storage_cfg)
        print(f"[worker] pulled {args.video_uri} -> {local_video}")

        # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
        video_id = compute_video_id(local_video)

        # 3. VALIDATE (same registry as the main CLI).
        val_results, val_context = validator_registry.run_all_with_context(local_video,
config)
        failures = [r for r in val_results if not r.passed]
        db = Database(os.path.join(scratch, config.output.db_name))
        db.add_video(video_id, local_video, "failed" if failures else "processed",
                     [r.model_dump() for r in val_results])

        segments: List[dict] = []
        segmenter_actual = None
        segmentation_ran = False
        status = "failed"
        try:
            if failures:
                print("[worker] validation FAILED: "
                      + "; ".join(f"{r.validator_name}: {r.message}" for r in failures))
                return 2
            seg_name = args.segmenter or config.indexing.default_segmenter
            segmenter_cls = segmenter_registry.get(seg_name)
            if not segmenter_cls:
                print(f"[worker] unknown segmenter: {seg_name!r} "
                      f"(have {list(segmenter_registry.list_available())})")
                return 2
            segmenter_actual = seg_name
            result = segmenter_cls(db, config).process_video(video_id, local_video,
max_frames=args.max_frames)
            segmentation_ran = True
            if isinstance(result, Failure):
                print(f"[worker] segmenter FAILED: {result.message}")
                return 3
            segments = result.value if hasattr(result, "value") else result
            status = "success"
        finally:
            # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
            emit_indexer_report(
                db=db, video_id=video_id, video_path=local_video, config=config,
                val_results=val_results, val_context=val_context,
                segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
                was_reingest=False, segmentation_ran=segmentation_ran,
```

```
            )

        # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
        out_local = os.path.join(scratch, "outputs", video_id)
        os.makedirs(out_local, exist_ok=True)
        _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
        _write_report_json(video_id, segments, status, os.path.join(out_local,
"report.json"))

        out_prefix = args.out_uri.rstrip("/")
        pushed = []
        for fn in sorted(os.listdir(out_local)):
            uri = f"{out_prefix}/outputs/{video_id}/{fn}"
            storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
            pushed.append(uri)

        print(f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):")
        for u in pushed:
            print("   ", u)
        return 0


    def _resolve_train_detector(args: argparse.Namespace, config: Any):
        """Build the trajectory DetectorRunner for `--trajectory-source detector`, honouring
        detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl: auto=WSL, native=GPU
box,
        stub=CPU mock). Returns the runner, or prints an error + returns None.

        Reuses the segmenter's `_resolve_runner` seam so the worker and the live CLI build
the
        detector identically (one source of truth). A non-trajectory segmenter (e.g. yolo)
has
        no shuttle detector and is rejected.
        """
        from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter

        seg_cls = segmenter_registry.get(args.segmenter)
        if not seg_cls:
            print(f"[worker] unknown segmenter: {args.segmenter!r} "
                  f"(have {list(segmenter_registry.list_available())})")
            return None
        seg = seg_cls(None, config)
        if not isinstance(seg, TrajectoryHybridSegmenter):
            print(f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
                  f"use a trajectory hybrid (wasb_hybrid / tracknet_hybrid /
fusion_hybrid).")
            return None
        runner, _ = seg._resolve_runner(config.get("indexing", {}))
        ok, msg = runner.healthcheck()
        if not ok:
            print(f"[worker] detector environment not ready: {msg}")
            return None
        print(f"[worker] detector ready ({args.segmenter}): {msg}")
        return runner


    def cmd_train(args: argparse.Namespace) -> int:
        """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest ->
shell out
        to training.gen0.harness (backend must NOT import training) -> push the artifact
bundle.

        Two trajectory sources (the train pipeline + harness are identical either way):
          * ``synth`` (default) ? the CPU "mock GPU": deterministic, label-consistent
```

```
synthetic
198             shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box /
CI.
199           * ``detector`` ? REAL shuttle trajectories: pull each video and run the resolved
200             DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test).
This
201             is the M3.5 "first real training" path; only the trajectory source flips.
202         """
203         import subprocess
204
205         config = load_config(args.config) if args.config else load_config()
206         _apply_overrides(config, args.set)
207         storage_cfg = config.storage.model_dump()
208
209         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
210         labels_dir = os.path.join(scratch, "labels")
211         traj_dir = os.path.join(scratch, "trajectories")
212         videos_dir = os.path.join(scratch, "videos")
213         os.makedirs(labels_dir, exist_ok=True)
214         os.makedirs(traj_dir, exist_ok=True)
215
216         corpus = args.corpus_uri.rstrip("/")
217         label_uris = sorted(u for u in storage.list_uris(f"{corpus}/labels/",
config=storage_cfg)
218                             if u.lower().endswith(".csv"))
219         if len(label_uris) < 2:
220             print(f"[worker] need >=2 labeled videos for leave-one-video-out; found
{len(label_uris)} "
221                   f"under {corpus}/labels/")
222             return 2
223
224         # Real-detector source: resolve the runner once + index the bucket's videos/ by
stem.
225         runner = None
226         video_by_stem: dict = {}
227         if args.trajectory_source == "detector":
228             os.makedirs(videos_dir, exist_ok=True)
229             runner = _resolve_train_detector(args, config)
230             if runner is None:
231                 return 2
232             for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
233                 vname = storage.parse_uri(vuri).name
234                 video_by_stem[os.path.splitext(vname)[0]] = vuri
235
236         print(f"[worker] trajectory source: {args.trajectory_source}")
237         specs: List[dict] = []
238         for uri in label_uris:
239             fname = storage.parse_uri(uri).name                   # e.g.
GX020094_proxy.rallies.csv
240             name = fname.replace(".rallies.csv", "").replace(".csv", "")
241             local_label = storage.fetch(uri, os.path.join(labels_dir, fname),
config=storage_cfg)
242             if args.trajectory_source == "detector":
243                 vuri = video_by_stem.get(name)
244                 if not vuri:
245                     print(f"[worker] no video for {name!r} under {corpus}/videos/ ?
skipping.")
246                     continue
247                 local_video = storage.fetch(vuri, os.path.join(videos_dir,
storage.parse_uri(vuri).name),
248                                             config=storage_cfg)
249                 video_id = compute_video_id(local_video)
250                 # Real fps/width drive the trajectory frame->time mapping (synth fixes them
at 30/1280).
251                 from backend.pipeline.segmenters.trajectory_hybrid import
```

```
TrajectoryHybridSegmenter
252                 fps, frame_width = TrajectoryHybridSegmenter._probe_fps_width(local_video)
253                 traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
254                 if not traj:
255                     print(f"[worker] detector produced no trajectory for {name!r} ?
skipping.")
256                     continue
257                 specs.append({"name": name, "trajectory": traj, "labels": local_label,
258                               "fps": fps, "frame_width": frame_width})
259             else:
260                 from backend.pipeline.detectors.stub_runner import
synth_trajectory_from_labels
261                 traj = os.path.join(traj_dir, f"{name}_traj.csv")
262                 synth_trajectory_from_labels(local_label, traj, fps=args.fps,
frame_width=args.frame_width)
263                 specs.append({"name": name, "trajectory": traj, "labels": local_label,
264                               "fps": args.fps, "frame_width": args.frame_width})
265
266         if len(specs) < 2:
267             print(f"[worker] need >=2 videos with trajectories for leave-one-video-out; got
{len(specs)}.")
268             return 2
269         manifest_path = os.path.join(scratch, "manifest.json")
270         with open(manifest_path, "w", encoding="utf-8") as f:
271             json.dump(specs, f, indent=2)
272         src_label = "real detector" if args.trajectory_source == "detector" else "CPU-mock
stub"
273         print(f"[worker] built manifest: {len(specs)} videos ({src_label} trajectories)")
274
275         out_dir = os.path.join(scratch, "artifacts")
276         cmd = [sys.executable, "-m", "training.gen0.harness",
277                "--manifest", manifest_path, "--out", out_dir, "--version", args.version]
278         if args.floor:
279             floor_local = (storage.fetch(args.floor, os.path.join(scratch, "floor.json"),
config=storage_cfg)
280                            if "://" in args.floor else args.floor)
281             cmd += ["--floor", floor_local]
282         print("[worker] training:", " ".join(cmd))
283         rc = subprocess.run(cmd).returncode
284         if rc != 0:
285             print(f"[worker] gen0 harness failed (rc={rc})")
286             return rc
287
288         # PUSH the versioned artifact bundle (weights.json + manifest.json) to the bucket.
289         bundle = os.path.join(out_dir, args.version)
290         out_prefix = args.out_uri.rstrip("/")
291         pushed = []
292         for fn in sorted(os.listdir(bundle)):
293             uri = f"{out_prefix}/weights/{args.version}/{fn}"
294             storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
295             pushed.append(uri)
296         print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
297         for u in pushed:
298             print("   ", u)
299         return 0
300
301
302     def build_parser() -> argparse.ArgumentParser:
303         p = argparse.ArgumentParser(prog="python -m backend.worker",
304                                     description="Storage-aware worker: pull ? run pipeline ?
push.")
305         sub = p.add_subparsers(dest="cmd", required=True)
306
307         pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
308         pi.add_argument("--video-uri", required=True, help="local path / local:// / s3:// /
```

```
gcs://")
309         pi.add_argument("--out-uri", required=True, help="output prefix
(outputs/<video_id>/? is appended)")
310         pi.add_argument("--segmenter", default=None, help="default:
indexing.default_segmenter")
311         pi.add_argument("--config", default=None, help="config.json path (default:
./config.json)")
312         pi.add_argument("--scratch", default=None, help="local scratch dir (default: a temp
dir)")
313         pi.add_argument("--max-frames", type=int, default=None)
314         pi.add_argument("--set", action="append", default=[], metavar="k=v",
315                         help="config override, e.g. --set indexing.skip_ai_handoff=true")
316         pi.set_defaults(func=cmd_infer)
317
318         pt = sub.add_parser("train", help="Gen-0 train-from-bucket (labels [+videos] ->
trajectories -> harness)")
319         pt.add_argument("--corpus-uri", required=True, help="prefix containing labels/ (+
videos/ for "
320                         "--trajectory-source detector), e.g. s3://bucket or gcs://bucket")
321         pt.add_argument("--out-uri", required=True, help="output prefix (weights/<version>/
is appended)")
322         pt.add_argument("--version", required=True, help="semver for the artifact bundle")
323         pt.add_argument("--trajectory-source", choices=["synth", "detector"],
default="synth",
324                         help="synth = CPU-mock label-consistent trajectories (default, no
GPU); "
325                              "detector = REAL shuttle trajectories via the resolved
DetectorRunner "
326                              "(needs videos/ in the corpus; honours detector_impl).")
327         pt.add_argument("--segmenter", default="wasb_hybrid",
328                         help="trajectory hybrid whose detector is used for --trajectory-
source detector")
329         pt.add_argument("--floor", default=None, help="floor baseline JSON (local path or
storage URI)")
330         pt.add_argument("--config", default=None)
331         pt.add_argument("--scratch", default=None)
332         pt.add_argument("--fps", type=float, default=30.0, help="synth trajectory fps
(detector probes the video)")
333         pt.add_argument("--frame-width", type=float, default=1280.0,
334                         help="synth trajectory frame width (detector probes the video)")
335         pt.add_argument("--set", action="append", default=[], metavar="k=v")
336         pt.set_defaults(func=cmd_train)
337         return p
338
339
340     def main(argv=None) -> int:
341         args = build_parser().parse_args(argv)
342         return args.func(args)
343
344
345     if __name__ == "__main__":
346         sys.exit(main())
347
```

6. assistant (2026-06-20T09:03:07.917Z)

Now let me read the key engine entry points and the existing UI proposal to ground the design precisely.

7. user (2026-06-20T09:03:09.358Z)

```
230             paths = emit_indexer_report(
231                 db=db_instance, video_id=video_id, video_path=req.video_path,
config=global_config,
232                 val_results=val_results, val_context=val_context,
```

```
                segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
                was_reingest=was_reingest, segmentation_ran=segmentation_ran,
            )
            if paths and isinstance(response, dict):
                response["reports"] = paths


    @app.post("/api/process", status_code=202)
    def process_video(req: ProcessRequest):
        """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.

        Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
        client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
        runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
        worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
        """
        allowed_root = getattr(getattr(global_config, "security", None),
"allowed_video_root", None)
        try:
            validate_input_path(req.video_path, allowed_root=allowed_root)
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e
        if not os.path.exists(req.video_path):
            raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
        if req.segmenter and not segmenter_registry.get(req.segmenter):
            available = list(segmenter_registry.list_available().keys())
            raise HTTPException(
                status_code=400,
                detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}"
            )

        job_id = uuid.uuid4().hex
        if not db_instance.create_job(job_id, req.video_path, req.segmenter,
                                      req.player_pool, req.max_frames):
            raise HTTPException(status_code=500, detail="Failed to persist job record.")
        _get_executor().submit(_drain_due_jobs)
        return JSONResponse(
            status_code=202,
            content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
        )


    @app.get("/api/jobs/{job_id}")
    def get_job(job_id: str):
        """Job state: {status, progress, result, reports}. `status` = execution state
        (queued|running|done|failed); the pipeline verdict is result["status"]."""
        job = db_instance.get_job(job_id)
        if not job:
            raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
        result = job.get("result")
        return {
            "job_id": job["id"],
            "status": job["status"],
            "progress": job.get("progress"),
            "video_id": job.get("video_id"),
            "error": job.get("error"),
            "result": result,
            "reports": (result or {}).get("reports"),
            "created_at": job.get("created_at"),
            "started_at": job.get("started_at"),
            "finished_at": job.get("finished_at"),
        }

```

```
    # --- Chunked upload (B2, PR-2) ----------------------------------------------------
    # The chunked-upload island (in-process registry + helpers + the 4 endpoints) now lives
in
    # backend.api.uploads as a self-contained APIRouter. It resolves the live startup config
via
    # backend.main.global_config, so behavior ? route paths, status codes, body-size limits,
    # sequential-part logic, and abort cleanup ? is unchanged.
    from backend.api import uploads as uploads_api

    app.include_router(uploads_api.router)


    # --- Highlight download (B3, PR-2) ------------------------------------------------

    @app.get("/api/highlights/{video_id}/{filename}")
    def download_highlight(video_id: str, filename: str):
        """Stream a compiled highlight reel ? `filename` is the bare name given to
/api/compile
        (which only writes into output_dir; the browser previously got back an unreachable
        server-local path)."""
        try:
            name = safe_output_filename(filename)
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e
        if not db_instance.get_video(video_id):
            raise HTTPException(status_code=404, detail="Indexed video record not found.")
        output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
or "output"
        path = os.path.join(output_dir, name)
        try:
            path = resolve_under_root(path, output_dir)
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e
        if not os.path.isfile(path):
            raise HTTPException(status_code=404,
                                detail=f"No compiled file named '{name}'. Compile first via
/api/compile.")
        media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
        return FileResponse(path, media_type=media, filename=name)


    @app.post("/api/query")
    def query_play_segments(req: QueryRequest):
        filters = {
            "server": req.server,
            "receiver": req.receiver,
            "duration_min": req.duration_min,
            "event_type": req.event_type
        }
        segments = db_instance.query_play_segments(req.video_id, filters)
        return {"play_segments": segments}

    @app.post("/api/compile")
    def compile_highlights(req: CompileRequest):
        video = db_instance.get_video(req.video_id)
        if not video:
            raise HTTPException(status_code=404, detail="Indexed video record not found.")

        # Retrieve matching play segments from db
        all_segments = db_instance.query_play_segments(req.video_id, {})
        matched_segments = [s for s in all_segments if s["id"] in req.segment_ids]

        if not matched_segments:
            raise HTTPException(status_code=400, detail="No matching play segments found to
stitch.")
```

```
354
355         # Reject path traversal / absolute output names (os.path.join would otherwise let an
356         # absolute or '..'-laden output_filename write anywhere on disk).
357         try:
358             out_name = safe_output_filename(req.output_filename)
359         except ValueError as e:
```

8. user (2026-06-20T09:03:09.953Z)

```
30
31      ## 3. ALPHA ? "quick & dirty" (goal: 10?25 invited testers, end-to-end, zero hand-
holding)
32
33      **Stack:** React + Vite + Tailwind SPA, deployed per HOSTING_PLAN (static hosting, $0).
34      API = existing FastAPI on the GPU box via tunnel at `api.khelsutra.guru`.
35      **Auth:** Cloudflare Access email-allowlist (OTP), free ?50 users, **zero auth code** ?
backend
36      reads the authenticated-email header for per-user scoping (alpha-grade tenancy: header
is
37      trustworthy because the tunnel only accepts Access-authenticated traffic).
38
39      ### Alpha screens (6)
40
41      | # | Screen | What it does | Backed by |
42      |---|--------|--------------|-----------|
43      | A1 | **Matches** (home) | User's videos, status chip per video (verdict ?/??/?),
upload CTA | `videos` + owner scoping (B4) |
44      | A2 | **Upload & Process** | Drag-drop; inline recording-guidelines checklist (pre-
upload education from VIDEO_RECORDING_GUIDELINES); chunked upload; then auto-starts processing |
B2 upload endpoint ? B1 job |
45      | A3 | **Job progress** | Stage list (validation ? segmentation ? AI confirm ? report)
with live status; verdict banner on completion | B1 `GET /api/jobs/{id}` poll |
46      | A4 | **Rally review** ? | Video player + rally list; click rally ? seek & play that
span; per-rally **Accept / Reject / nudge start?end ±0.5 s**; bulk "all good"; chips: duration,
shot count (+confidence) | B6 correction endpoints ? `candidate_segments.human_label/notes` ?
**the corpus engine** |
47      | A5 | **Highlights** | Select accepted rallies ? name reel ? compile ? download; list
of past reels | `/api/compile` + B3 download endpoint |
48      | A6 | **Report** | Renders the customer-audience report (verdict, notes, FAQ refs);
"nerd view" toggle shows technical.md | existing reporting artifacts |
49
50      ### Backend work the alpha actually requires (honest dependency list)
51
52      | ID | Item | Notes |
53      |----|------|-------|
54      | B1 | **Async job model ? DEFERRED_HARDENING_PLAN #16** | The committed next PLATFORM
slice; `POST /api/process` ? 202 + `job_id`, `jobs` table (user_version 1?2),
ThreadPoolExecutor, `GET /api/jobs/{id}`. **Approving this proposal = greenlighting #16.** A
5?60 min blocking HTTP call cannot sit behind a tunnel UI. |
55      | B2 | **Chunked upload endpoint** | `POST /api/upload` accepting ?90 MB parts (free-
tier proxy body limit is 100 MB), assemble ? MD5 ? register under `allowed_video_root/<user>/`.
Size cap (e.g. 4 GB) + mp4/mov allowlist. |
56      | B3 | **Download endpoint** | `GET /api/highlights/{video_id}/{name}` streaming the
compiled reel (today `/api/compile` returns a server-local path the browser can't reach). |
57      | B4 | **Per-user scoping** | `owner_email` column on `videos` (migration), read from
the Access header, filter every query. Alpha-grade, single DB. |
58      | B5 | **Retire/replace static UI** | The SPA supersedes `backend/api/static` (which has
a known `rallies` vs `play_segments` field bug ? don't fix, replace). |
59      | B6 | **Correction-loop endpoints** | `PATCH /api/segments/{id}` ?
accept/reject/boundary-nudge persisted to `candidate_segments.human_label`/`notes` (+
provenance: who, when). This is v1-scope "correction loop" made real. |
60      | B7 | **Exposure hardening** | CORS narrowed to the app origin; basic rate limit;
upload quota per user; `allowed_video_root` set; Gemini key rotated (already in NEXT_STEPS
housekeeping). |
61
```

```
62      ### Alpha non-goals (explicit)
63
64      No payments, no orgs/teams, no athlete profiles, no share links, no owned-model serving,
no
65      multi-sport selector (badminton only), no notifications, no mobile-perfect layouts
(usable on
66      phone, optimized for laptop), no conversational QA (deferred in roadmap).
67
68      ### Alpha effort estimate
69
```

9. assistant (2026-06-20T09:03:14.784Z)

I have enough grounding now. The research bundle is extremely detailed and I've verified the load-bearing facts (worker CLI flow, the FastAPI loop, the storage identity passthrough, the no-auth/CORS-* reality, the static UI gaps, the M3.5 native-runner gap). Let me check one more thing ? whether there's a way to launch the worker as a daemon vs only as a one-shot CLI, since that's the crux of "one UI, compute placement is a setting."

10. user (2026-06-20T09:03:16.294Z)

```
211:        # re-claims it via `claim_due_job` when due ? with NO master node: the table IS
486:    def create_job(self, job_id: str, video_path: str, segmenter: Optional[str] = None,
561:        worker. `claim_due_job` re-picks it when due ? cooperative, no master.
``next_poll_at`` is
588:    def claim_due_job(self) -> Optional[Dict[str, Any]]:
```

11. assistant (2026-06-20T09:03:23.461Z)

I have all I need. The jobs table stores `video_path` (a local path), the FastAPI in-process worker (`max_workers=1`) drains it, and the decoupled `python -m backend.worker` is a separate one-shot CLI reading from a StorageBackend URI. The crux of the design is reconciling these two: the FastAPI loop (local path, in-process) vs the decoupled worker (URI, separate process/box).

The honest single-box design: keep the FastAPI loop as the UI's control plane and data plane; make the local-filesystem case the degenerate `storage.backend='local'` config; and make "remote GPU" a settings-driven swap to the decoupled worker reading the same bucket ? without pretending the bucket-driven worker path is wired to FastAPI today (it is not; that's net-new orchestration).

Let me deliver the structured design.

12. user (2026-06-20T09:04:53.794Z)

```
<tool_use_error>InputValidationError: StructuredOutput was called with input that could not be
parsed as JSON.
You sent (first 200 of 16129 bytes): {"name": "KhelSutra Studio ? single-box rally indexer +
reel console (decoupled-profile appliance)", "one_liner": "One web console (the web/ SPA, served
behind Cloudflare Access) that drives the exist
Common causes: unescaped backslashes in file paths (use / or \\), unescaped control characters,
or truncated output. Retry with valid JSON.</tool_use_error>
```

13. assistant (2026-06-20T09:04:56.369Z)

The input got wrapped oddly. Let me call the tool with proper structured fields directly.

14. user (2026-06-20T09:06:18.248Z)

```
<tool_use_error>InputValidationError: StructuredOutput was called with input that could not be
parsed as JSON.
You sent (first 200 of 15281 bytes): {"name": "KhelSutra Studio ? single-box rally indexer +
reel console (decoupled-profile appliance)", "one_liner": "One web console (the khelsutra web/
SPA, served behind Cloudflare Access) drives the
Common causes: unescaped backslashes in file paths (use / or \\), unescaped control characters,
```

or truncated output. Retry with valid JSON.</tool_use_error>

15. assistant (2026-06-20T09:06:22.860Z)

The harness is mangling my JSON-in-a-string. I should pass the structured fields as native arguments, not as a pre-serialized JSON blob. Let me write the content to a file first so I can construct it cleanly, then pass it as proper structured arguments.

16. user (2026-06-20T09:07:44.301Z)

<tool_use_error>InputValidationError: StructuredOutput was called with input that could not be parsed as JSON.
You sent (first 200 of 15404 bytes): {"name": "KhelSutra Studio ? single-box rally indexer + reel console (decoupled-profile appliance)", "one_liner": "One web console (the khelsutra web/ SPA, served behind Cloudflare Access) drives the
Common causes: unescaped backslashes in file paths (use / or \\), unescaped control characters, or truncated output. Retry with valid JSON.</tool_use_error>

17. assistant (2026-06-20T09:07:51.723Z)

The issue is the em-dash / unicode and the way I'm structuring it. The tool expects native object fields, not a `{"raw": "..."}` wrapper. Let me build the JSON in a file with Python to guarantee validity, then read it back and pass each field. Actually ? the cleanest path is to call StructuredOutput with the actual object keys directly. Let me write a Python script to produce a validated JSON object and print it, so I can copy exact field values, avoiding any manual escaping.

18. user (2026-06-20T09:09:15.476Z)

Exit code 2
/usr/bin/bash: -c: line 16: unexpected EOF while looking for matching `''

19. assistant (2026-06-20T09:09:17.950Z)

The heredoc broke on the apostrophe in `worker's`/`box's`. Let me write the Python file directly with the Write tool instead, which avoids shell quoting entirely.

20. assistant (2026-06-20T09:10:20.369Z)

```
import json

obj = {
  "name": "KhelSutra Studio - single-box rally indexer + reel console (decoupled-profile appliance)",
  "one_liner": "One web console (the khelsutra web/ SPA, served behind Cloudflare Access) drives the existing FastAPI engine through its full upload->index->pick-rallies->reel->download loop on ONE box, where compute placement is a settings choice: the SAME UI watches job/output state whether rallies come from gemini/stub on the CPU droplet, native-WASB on a co-located GPU, or a remote GPU worker draining the shared bucket.",

  "topology": "ONE box runs four cooperating things, mirroring the droplet today (168.144.126.176 already co-hosts /srv/khelsutra-site + ~/rally). (1) The khelsutra web/ SPA (Vite+React+TS+Tailwind, Cloudflare Pages build root) is the operator console - static assets, zero engine code. (2) The engine FastAPI 'python -m backend.main --server' on 127.0.0.1:8000 is BOTH the control plane (the shipped jobs table + ThreadPoolExecutor(max_workers=1) self-scheduling drain in job_worker.py; the DB jobs table IS the clock, no central scheduler) AND the data plane (uploads.py chunked upload, /api/compile, /api/highlights FileResponse stream). (3) Storage = the StorageBackend ABC; on a single box it is the DEFAULT storage.backend='local' (config/models.py:298), where fetch_to_local is an identity passthrough (local.py:33) and put no-ops on src==dst (local.py:42-47), so a worker pull->pipeline->push collapses to read-local -> run -> write output_dir, byte-for-byte the pre-storage behaviour. (4) Compute substrate = the detector_impl seam in trajectory_hybrid.py::_resolve_runner: gemini (cloud, no GPU), stub (CPU mock), auto=WSL, native=GPU. HONESTY: today the UI loop runs ENTIRELY through the in-process FastAPI worker - the jobs table stores a local video_path, NOT a storage URI - and the decoupled
```

'python -m backend.worker {infer|train}' is a SEPARATE one-shot CLI the FastAPI does NOT call. So the single-box MVP is the FastAPI loop with storage=local + segmenter=gemini-or-stub; the bucket worker is the same box SECOND mode for the BYO-GPU story, reached by config, not by a new always-on daemon (the UI<->bucket-worker bridge is net-new, deferred past MVP).",

"request_path": "ONE origin in the MVP, TWO in the gated topology. MVP (single box): the web/ SPA is served as static assets; all data calls hit the same engine FastAPI on :8000 (Vite proxy in dev, the HOSTING_PLAN tunnel in prod). A browser request reaches the engine as: SPA -> POST /api/process {video_path} -> 202 {job_id} -> poll GET /api/jobs/{id} every 3s until done|failed -> POST /api/query -> POST /api/compile -> GET /api/highlights/{video_id}/{filename} (FileResponse). SOURCE/REEL VIDEO is served by the engine box itself: the reel streams from /api/highlights; for review playback there is NO endpoint today (gap - see ui_console). The GATED prod topology (HOSTING_PLAN.md:24-44) splits app.khelsutra.guru (Cloudflare Pages -> web/ SPA) from api.khelsutra.guru (Cloudflare Tunnel -> FastAPI :8000), both behind Cloudflare Access email-OTP; that is two hostnames but still ONE engine box. Raw 168.144.126.176:8000 today has CORS '*' and zero auth - the tunnel+Access edge is the ONLY tenancy boundary, never the app.",

"compute_placement": "THREE placements, ONE UI, expressed as engine config the console surfaces from GET /api/config (it already returns the live typed AppConfig) - NOT three code paths in the UI. (A) CPU droplet (gemini or stub): segmenter='gemini' needs GEMINI_API_KEY, no GPU, uploads <=1280px chunks to Google (state in consent copy); OR a trajectory hybrid with RALLY_DETECTOR_IMPL=stub + indexing.skip_ai_handoff=true = the GPU-free, key-free CPU mock (the validated droplet e2e). Runs through the in-process FastAPI worker - SHIPPED, this is the MVP. (B) Co-located GPU box (native WASB): segmenter='wasb_hybrid' + indexing.detector_impl='native' -> NativeWasbRunner. BLOCKED: the base _build_native_runner REFUSES with NotImplementedError (trajectory_hybrid.py:57-63); native_wasb_runner.py is the M3.5 NEXT build, not landed. On Windows+WSL, detector_impl='auto' gives the REAL WSL detector today. Still runs through the in-process FastAPI worker once the runner exists. (C) Remote GPU worker via bucket (BYO-compute): storage.backend='s3'/'gcs' + 'python -m backend.worker infer --video-uri s3://.../videos/<md5>.mp4 --out-uri s3://...' on a separate GPU droplet; the UI watches outputs/<video_id>/ in the bucket instead of polling /api/jobs. The SAME web/ SPA works because the per-rally screen needs only {intervals, report, reel} - but the orchestration that enqueues a bucket job and reflects its state into the UI is NET-NEW (no /api/jobs over storage exists). MVP picks (A); (B)/(C) are the same console with compute as a setting, gated on M3.5 + the bridge.",

"auth_exposure": "Edge auth, NOT app auth - and say so loudly. The engine FastAPI ships allow_origins=['*'], allow_credentials=False, and ZERO auth dependency (main.py:42-46); a publicly-reachable :8000 is wide open. The single intended gate is Cloudflare Access email-OTP allowlist (<=50 alpha testers) in front of BOTH app.khelsutra.guru and api.khelsutra.guru, reached via a Cloudflare Tunnel (cloudflared, outbound-only, no inbound ports, home/box IP never exposed - HOSTING_PLAN.md:36). The backend trusts the Cf-Access-Authenticated-User-Email header for per-user scoping (alpha-grade tenancy - trustworthy ONLY because the tunnel accepts only Access-authenticated traffic). For the BYO-compute single-tenant case the recommended posture is: bind the engine to localhost + reach it via the Access-gated tunnel (no token in the app), and supply a single shared per-tenant token ONLY if a non-Cloudflare reverse proxy is used. Go-live checklist that does NOT exist yet and MUST precede exposure: Access allowlist ON, CORS narrowed to https://app.khelsutra.guru, basic rate limit + per-user upload quota, security.allowed_video_root jail set, GEMINI_API_KEY rotated (it was chat-shared). Treat the gate, not the app, as the boundary until app-layer auth (P1) lands.",

"ui_console": "Six screens in the web/ SPA; the MVP is the four that ride EXISTING endpoints with zero new engine code, embedding the already-approved RallyPicker as the curation screen. (S1) Upload: a file-picker that chunks <=max_part_mb via /api/upload/init|part|complete (these endpoints EXIST but app.js never calls them - the net-new client wiring), then feeds the returned absolute path into POST /api/process. Inline recording-guidelines checklist. (S2) Process/Progress: shows the coarse free-text stage from GET /api/jobs/{id} (hashing->validating->segmenting->finalizing) on a 3s poll + a 'one job at a time' notice (max_workers=1) and an honest 'failed: interrupted by server restart' state (fail_stale_jobs sweeps queued/running at startup). No %/ETA/SSE today. (S3) Rally index + selection = EMBED docs/PER_RALLY_SELECTION.md RallyPicker verbatim: POST /api/query -> select segment_ids -> deterministic query bar (server/receiver/duration/event), honouring GET /api/config min_shot_count; NEVER render motion-only server/receiver/events or a confidence meter from the Gemini string. (S4) Reels/History: name reel -> POST /api/compile -> GET /api/highlights download link. (S5) Settings = compute+storage: read GET /api/config to SHOW the segmenter

registry + default + detector_impl + whether a GPU/key is present, and let the user pick gemini vs wasb_hybrid vs stub + toggle skip_ai_handoff; storage backend shown read-only in MVP. (S6) Library/Matches: list all videos+verdict chips and a reels gallery. GAPS the console must own or stub honestly: NO GET /api/videos, NO GET /api/jobs list, NO list-reels, NO review-playback stream, NO cancel-job - S6 + reel gallery + review playback need net-new engine endpoints (declare them as gated deliverables, do not fake them).",

  "reuse_vs_new": "REUSES (do not re-plumb): the StorageBackend ABC + URI scheme (backend/storage/, local default = identity) and the worker dispatcher backend/worker/__main__.py (pull->SAME pipeline blocks->push, a thin orchestrator never a fork); the entire FastAPI loop (process/jobs/query/compile/highlights + the chunked-upload endpoints, which exist but are unwired in app.js); the jobs table + max_workers=1 self-scheduling drain (job_worker.py) as the control plane; the per-rally doc docs/PER_RALLY_SELECTION.md (embed its RallyPicker + reuse its verified engine-contract anchors S3/S4 verbatim, adopt its honesty discipline); the khelsutra web/ Vite+React scaffold + its useRallySelection()/typed-API-client seams (P1-P2); the host-portability adapter shape (handleRequest(request,store) over d1Store|sqliteStore) as precedent for the one-URI storage contract; HOSTING_PLAN.md Access+Tunnel topology as the ready-made gate; the parity.yml/test.yml green-CI-gates-merge discipline; docs/PROJECT_DOC_TEMPLATE.md for the tracked doc. NET-NEW: the upload client wiring in the SPA; the Settings (compute+storage) screen reading GET /api/config; GET /api/videos (+owner scoping), GET /api/jobs (list/history), list-compiled-reels, and a review-playback stream endpoint; and the deferred UI<->bucket-worker bridge (an endpoint that enqueues 'python -m backend.worker' against a bucket and reflects outputs/<video_id>/ state into the same jobs/poll contract) plus the M3.5 NativeWasbRunner for real co-located GPU.",

  "mvp_scope": [
    "Single box, storage.backend='local' (default), segmenter='gemini' (config.json default) OR stub+skip_ai_handoff for a key-free CPU run - the validated droplet path; NO new engine code for the core loop.",
    "Wire the EXISTING chunked-upload endpoints (/api/upload/init|part|complete) into the web/ SPA file-picker, then feed the returned absolute path into POST /api/process (the one true client gap blocking browser upload).",
    "Drive the loop end-to-end: upload -> POST /api/process (202+job_id) -> poll GET /api/jobs/{id} with coarse stage text + a 'one job at a time' notice -> embed RallyPicker (POST /api/query -> segment_ids) -> POST /api/compile -> GET /api/highlights download. (S1-S4.)",
    "Settings screen: render GET /api/config to show the segmenter registry, default, detector_impl, and whether a GPU/Gemini key is present; allow choosing segmenter + toggling skip_ai_handoff per /api/process request. (S5 - compute placement is a visible setting even before remote GPU is wired.)",
    "Serve behind the HOSTING_PLAN Access+Tunnel gate; complete the go-live hardening (CORS narrow, allowed_video_root, rate limit/quota, rotate Gemini key) BEFORE any public exposure.",
    "Ship as a tracked doc in khelsutra (peer-of PER_RALLY_SELECTION.md, PROJECT_DOC_TEMPLATE structure, indexed in docs/README.md + a NEXT_STEPS pointer), one small PR per S-row off origin/master, no stacking, green CI gates merge."
  ],

  "pros": [
    "Fastest real CUJ: the core loop already exists in FastAPI; the dominant net-new work is client-side upload wiring + reusing the approved RallyPicker, so a single box delivers upload->reel without new engine code.",
    "Honest 'compute is a setting': GET /api/config already exposes the registry/default/detector_impl, so Settings surfaces gemini/stub/native truthfully and the SAME UI later points at a bucket worker - the BYO-compute story the request-access form probes, with no UI rewrite.",
    "Reconciles cleanly with the decoupled model: storage=local is the degenerate StorageBackend config and the in-process worker is the degenerate ComputeBackend='local', so nothing fights the StorageBackend/worker design - the single box is literally the collapsed three-plane split.",
    "Privacy-forward default: stub+skip_ai_handoff runs with NO key and NO bytes to Google; gemini is an explicit opt-in toggle, matching the L0->L2 locality ladder and consent guardrails.",
    "Operationally proven: the exact box (DO droplet) already co-hosts site+engine and ran the full train+infer e2e against real Spaces with the GPU mocked - this design is the UI on top of a verified substrate."
  ],

```
  "cons": [
    "max_workers=1 ceiling: strictly one job at a time; a 2nd submit queues with no
position/ETA, and a server restart fails in-flight jobs - the UI must communicate this, it
cannot hide it.",
    "No real GPU on the CPU droplet: real (non-mock) detection there is impossible; you get
gemini (cloud, costs + leaks chunks) or stub (mock, not real rallies). Real local GPU =
Windows+WSL (auto) today; a Linux GPU box needs the unbuilt M3.5 NativeWasbRunner.",
    "The decoupled bucket-worker path is invisible to the UI: driving 'python -m backend.worker'
from the console (placement C) needs a net-new orchestration bridge that does not exist - so
'remote GPU as a setting' is design, not shipped.",
    "Engine has zero auth + CORS '*': the appliance is only as safe as the Cloudflare Access
edge; a misconfigured/bypassed gate exposes everything, including unauthenticated uploads.",
    "Review playback + retention gaps: there is no endpoint to stream the source/proxy for in-UI
rally review (S3 can list+select but not play-the-span), and uploaded originals are never
deleted today - both must be addressed before broad exposure."
  ],

  "honest_caveats": [
    "[verified] The UI working loop runs through the in-process FastAPI worker (jobs table
stores a local video_path); the decoupled 'python -m backend.worker {infer|train}' is a separate
one-shot CLI the FastAPI does NOT invoke. 'One UI for both' is true for storage=local + the
visible segmenter setting; the bucket-worker bridge is net-new and out of MVP.",
    "[verified] detector_impl='native' REFUSES today - base _build_native_runner raises
NotImplementedError (trajectory_hybrid.py:57-63); native_wasb_runner.py is the M3.5 next build.
Co-located real GPU on Linux is NOT available until it lands; on Windows the WSL 'auto' path
works.",
    "[verified] The CPU droplet has NO GPU; the only GPU-free real segmenter is 'motion'.
wasb/tracknet/fusion need a GPU+WSL/native; 'gemini' needs GEMINI_API_KEY and uploads <=1280px
chunks to Google even in otherwise-local modes - must be in consent copy.",
    "[verified] Engine FastAPI = no auth, CORS '*'; Cloudflare Access (edge) is the ONLY tenancy
boundary in alpha. Per-user scoping is an unbuilt owner_email column; multi-tenant identity/RLS
is P1.",
    "[verified] Client gaps in app.js: the chunked-upload endpoints exist but are never called;
there is no GET /api/videos, no GET /api/jobs list, no list-reels, no review-playback stream, no
cancel; progress is coarse free-text on a 3s poll (no %/ETA/SSE). The console must add these as
net-new (engine endpoints for the list/playback ones) or stub them honestly.",
    "[verified] Uploaded originals under security.upload_dir are never deleted (retention is an
open gap); a publicly-reachable single box accumulates many users' video - default to keeping
proxies+intervals and add retention before broad exposure.",
    "[design] This is a design only; nothing here is implemented. It should be written up as a
tracked doc (peer-of PER_RALLY_SELECTION.md / REQUEST_ACCESS_FORM.md) with a S7 progress tracker
as the source of truth, every claim tagged [verified]/[researched]/[design], and is owner-gated
before any build."
  ]
}

with open(r"C:\Users\avidu\Projects\badminton-highlight-indexer\artifacts\so.json", "w",
encoding="utf-8") as f:
    json.dump(obj, f)
print("VALID", len(json.dumps(obj)))
```

21. user (2026-06-20T09:10:20.383Z)

```
File created successfully at: C:\Users\avidu\Projects\badminton-highlight-
indexer\artifacts\so.py (file state is current in your context ? no need to Read it back)
```

22. user (2026-06-20T09:10:25.514Z)

```
VALID 15148
KEYS ['auth_exposure', 'compute_placement', 'cons', 'honest_caveats', 'mvp_scope', 'name',
'one_liner', 'pros', 'request_path', 'reuse_vs_new', 'topology', 'ui_console']
OK
```

23. assistant (2026-06-20T09:10:27.802Z)

The JSON is valid and has all 12 keys. Now I'll pass it to StructuredOutput as the actual structured object (not a `raw` wrapper).

24. user (2026-06-20T09:11:32.039Z)

Structured output provided successfully

25. assistant (2026-06-20T09:11:44.347Z)

Design delivered via StructuredOutput.

Summary of the design ("KhelSutra Studio"): the smallest buildable slice is the existing khelsutra `web/` SPA driving the engine's existing FastAPI loop on ONE box with `storage.backend='local'` (the default identity-passthrough) and `segmenter=gemini` or `stub+skip_ai_handoff` (the GPU-free, key-free CPU path already validated on the droplet). The only true client gap to close for the full CUJ is wiring the already-built `/api/upload/*` chunked endpoints into the SPA, then feeding the returned path into the existing `/api/process -> /api/jobs poll -> /api/query -> /api/compile -> /api/highlights` chain, with the approved RallyPicker embedded as the curation screen.

The load-bearing honesty points I grounded against the code:
- The UI loop runs through the **in-process FastAPI worker** (`jobs` table stores a local `video_path`), NOT the decoupled `python -m backend.worker` (a separate one-shot CLI reading storage URIs the FastAPI never invokes). "One UI, compute is a setting" is honest only for `storage=local` + the visible segmenter setting; the UI?bucket-worker bridge for remote-GPU placement is net-new and deferred.
- `detector_impl='native'` **refuses today** (`trajectory_hybrid.py:57-63` raises `NotImplementedError`); `native_wasb_runner.py` is the unbuilt M3.5 piece, so real co-located Linux GPU is not available yet.
- Engine has **zero auth + CORS '*'**; Cloudflare Access (edge) is the only tenancy boundary.

Artifacts written for the run: `C:\Users\avidu\Projects\badminton-highlight-indexer\artifacts\so.py` and `artifacts\so.json` (the validated payload) ? both under the already-gitignored `artifacts/` dir; safe to delete.